[COLLEGE / INSTITUTION NAME]
Department of Computer Engineering

A MINI PROJECT REPORT ON
Cloud Computing Lab (CCL) Mini Project Report

SwachhNet: AI-Powered Smart Garbage Reporting and Municipal Dispatch System

A Cloud-Native Municipal Solid Waste Management Platform Built on Amazon Web Services (AWS)

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Engineering in Computer Engineering

SUBMITTED BY:
Kamesh Nerpagar (Roll No: [ROLL NO. 1])
[STUDENT 2 NAME] (Roll No: [ROLL NO. 2])

UNDER THE GUIDANCE OF:
Prof. [GUIDE NAME]
Assistant Professor, Department of Computer Engineering

Academic Year: 2026 – 2027
Mumbai, Maharashtra, India

CERTIFICATE

This is to certify that the mini-project report entitled "SwachhNet: AI-Powered Smart Garbage Reporting and Municipal Dispatch System" is a bona fide work carried out by Kamesh Nerpagar (Roll No: [ROLL NO. 1]) and [STUDENT 2 NAME] (Roll No: [ROLL NO. 2]) in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Computer Engineering in Cloud Computing Laboratory during the academic year 2026–2027.

The project has been executed under our supervision and guidance, and the infrastructure has been deployed, tested, and verified on Amazon Web Services (AWS).

_____	_____	_____
Prof. [GUIDE NAME]	Dr. [HOD NAME]	External Examiner
Project Guide	Head of Department	College Seal

DECLARATION

We hereby declare that the mini-project entitled "SwachhNet: AI-Powered Smart Garbage Reporting and Municipal Dispatch System" submitted for the Cloud Computing Laboratory examination is our original work conducted under the guidance of Prof. [GUIDE NAME].

The information and empirical findings presented in this report represent actual software implementations, database schemas, and cloud infrastructure provisioned in the Amazon Web Services (AWS) Asia Pacific (Mumbai) region (ap-south-1). Any material or reference drawn from external literature, documentation, or cloud providers has been properly cited and acknowledged.

Date: 30th October 2026
Place: Mumbai

Kamesh Nerpagar (Roll No: [ROLL NO. 1])
[STUDENT 2 NAME] (Roll No: [ROLL NO. 2])

ACKNOWLEDGEMENT

We express our deepest gratitude to our project guide, Prof. [GUIDE NAME], for their constant encouragement, invaluable suggestions, and constructive guidance throughout the duration of this Cloud Computing Lab project.

We are also thankful to the Head of the Department of Computer Engineering and the faculty members of the Cloud Computing Laboratory for providing state-of-the-art laboratory facilities and technical support. Finally, we thank our peers and families for their continuous motivation and support during the design, cloud deployment, and documentation phases of this project.

ABSTRACT

Rapid urbanization and escalating municipal solid waste generation pose critical operational challenges for metropolitan governance bodies like the Brihanmumbai Municipal Corporation (BMC). Traditional grievance redressal mechanisms are predominantly centralized, manually mediated, lack automated spatial triage, and exhibit poor accountability due to absence of verifiable physical evidence.

This project presents SwachhNet, an enterprise-grade, cloud-native municipal solid waste reporting and automated dispatch platform architected on Amazon Web Services (AWS). SwachhNet facilitates end-to-end municipal grievance handling: citizens report localized garbage dumps with real-time GPS coordinates and camera captures; multimodal artificial intelligence (Google Gemini Vision) autonomously audits photographs to reject fraudulent submissions; geospatial Haversine algorithms dynamically partition and route complaints to the appropriate administrative ward; and automated dispatch mechanisms assign the nearest idle municipal garbage truck. Upon sanitation completion, workers submit photographic evidence, which undergoes before-and-after visual differential verification before a complaint is marked resolved.

To satisfy the requirements of the Cloud Computing Laboratory (CCL) curriculum, the system underwent a complete cloud transformation from a monolithic local architecture into a decoupled, highly secure, and cost-optimized AWS infrastructure deployed in the Mumbai (ap-south-1) region. The production cloud architecture features: (1) a custom Amazon VPC (10.0.0.0/16) with segregated public and multi-AZ private subnets; (2) Amazon EC2 (t3.micro) compute running Nginx as a reverse proxy and Express.js daemonized via PM2; (3) Amazon RDS for PostgreSQL 16.10 (db.t3.micro) housed within an isolated private DB subnet group with zero public internet exposure; (4) Amazon S3 private object storage with AES-256 server-side encryption and 60-minute presigned URL security; (5) AWS IAM instance profiles enforcing least-privilege security; (6) AWS Systems Manager (SSM) Parameter Store for centralized encrypted configuration; (7) Amazon SQS with a Dead Letter Queue (DLQ) for asynchronous background task processing; (8) Amazon CloudWatch for centralized application log streaming and metric alarms; (9) AWS Budgets enforcing a strict $10.00/month cost ceiling; and (10) AWS CloudFormation delivering single-click Infrastructure as Code (IaC) deployment. The system is fully operational at public endpoint http://65.2.180.175, achieving sub-second API latencies, zero database internet exposure, and 100% automated municipal dispatching.

TABLE OF CONTENTS

Preliminary Pages

Certificate
Declaration
Acknowledgement
Abstract
List of Abbreviations
List of Figures
List of Tables

Chapter 1: Introduction

1.1 Introduction to Cloud Computing
1.2 Introduction to SwachhNet
1.3 Background and Motivation
1.4 Problem Statement
1.5 Existing System Analysis
1.6 Limitations of the Existing System
1.7 Proposed Cloud-Based System
1.8 Project Objectives
1.9 Scope of the Project
1.10 Advantages of the Proposed Cloud Architecture

Chapter 2: Literature Review & Technology Overview

2.1 Overview of Modern Cloud Architecture
2.2 Frontend Framework (React 18 & Vite)
2.3 Backend Runtime & Framework (Node.js & Express.js)
2.4 Managed Relational Database (Amazon RDS PostgreSQL)
2.5 Virtual Compute Infrastructure (Amazon EC2)
2.6 Private Cloud Isolation (Amazon VPC)
2.7 Secure Object Storage (Amazon S3)
2.8 Identity & Access Management (AWS IAM)
2.9 Configuration Management (AWS SSM Parameter Store)
2.10 Asynchronous Messaging & Buffering (Amazon SQS)
2.11 Telemetry & Monitoring (Amazon CloudWatch)
2.12 Financial Governance (AWS Budgets)
2.13 Infrastructure as Code (AWS CloudFormation)
2.14 Artificial Intelligence (Google Gemini Vision API)
2.15 Token Security & Geospatial Libraries (JWT & Leaflet)

Chapter 3: System Analysis & Requirements

3.1 Functional Requirements
3.2 Non-Functional Requirements
3.3 Hardware Requirements (Development & Production)
3.4 Software Requirements

3.5 Cloud Infrastructure Requirements
3.6 User Roles & Privilege Matrix
3.7 Feasibility Study (Technical, Economic, Operational)
3.8 Cloud Security & Boundary Requirements

Chapter 4: System Architecture & Design

4.1 Overall SwachhNet Cloud Architecture
4.2 Amazon VPC Network & Subnet Topography
4.3 Multi-Tier Software Architecture
4.4 Relational Database Design (PostgreSQL Schema & ERD)
4.5 UML Use Case Modeling
4.6 Data Flow Modeling (Level 0 Context & Level 1 DFD)
4.7 Core Municipal Workflows (Complaint, Dispatch, Cleanup, SLA)

Chapter 5: AWS Cloud Services Integration

5.1 Amazon Virtual Private Cloud (VPC)
5.2 Amazon Elastic Compute Cloud (EC2)
5.3 Amazon Relational Database Service (RDS for PostgreSQL)
5.4 Amazon Simple Storage Service (S3)
5.5 AWS Identity and Access Management (IAM)
5.6 AWS Systems Manager (SSM) Parameter Store
5.7 Amazon Simple Queue Service (SQS) & DLQ
5.8 Amazon CloudWatch Monitoring & Metrics
5.9 AWS Budgets & Cost Governance
5.10 AWS CloudFormation (Infrastructure as Code)
5.11 Comprehensive Service Mapping & Curriculum Compliance

Chapter 6: Implementation & Working

6.1 Frontend User Interface Implementation
6.2 Backend Dual-Database Engine Bootstrap
6.3 Authentication & Role-Based Middleware
6.4 Database Pool, Migrations & Transaction Helpers
6.5 Amazon S3 Evidence Upload & Presigned URL Resolution
6.6 Multimodal AI Visual Verification Engine
6.7 Geospatial Ward Partitioning (Haversine SQL)
6.8 Automated Nearest Truck Dispatch Mechanism
6.9 Sanitation Worker Task Life Cycle
6.10 Before-and-After Verification Algorithm
6.11 Citizen Reward Points & Badging Gamification
6.12 48-Hour SLA Breach Background Evaluation
6.13 Asynchronous Worker Polling via Amazon SQS
6.14 CloudWatch Structured Logging with PII Sanitization

Chapter 7: Deployment & Configuration

7.1 Pre-Deployment Prerequisites & IAM Credentials
7.2 Declarative Infrastructure Provisioning (CloudFormation)

7.3 Parameter Store Secret Population
7.4 Amazon EC2 Host Bootstrapping & Nginx Configuration
7.5 Database Migrations & BMC Seed Execution
7.6 PM2 Process Daemonization & Startup Hooks
7.7 Production Verification & Health Probes

Chapter 8: Testing & Results

8.1 Testing Methodology
8.2 Comprehensive Test Suite (20 Test Scenarios)
8.3 Empirical Test Execution Results
8.4 Cloud Evidence & Console Verification Gallery

Chapter 9: Cost Analysis & Cloud Security

9.1 Monthly Cloud Infrastructure Cost Breakdown
9.2 AWS Free Tier Utilization & Optimization Strategies
9.3 Multi-Layered Defense-in-Depth Security Architecture

Chapter 10: Conclusion & Future Scope

10.1 Project Summary & Achievements
10.2 Academic & Practical Cloud Principles Demonstrated
10.3 Limitations of Current Implementation
10.4 Future Roadmap & Enhancements

References

Academic & Professional Citations

LIST OF ABBREVIATIONS

Abbreviation	Expanded Description
AWS	Amazon Web Services
VPC	Virtual Private Cloud
EC2	Elastic Compute Cloud
RDS	Relational Database Service
S3	Simple Storage Service
IAM	Identity and Access Management
SSM	Systems Manager
SQS	Simple Queue Service
DLQ	Dead Letter Queue
IGW	Internet Gateway
CIDR	Classless Inter-Domain Routing
BMC	Brihanmumbai Municipal Corporation
SLA	Service Level Agreement
SPA	Single Page Application
API	Application Programming Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
IaC	Infrastructure as Code
SSE	Server-Side Encryption
URL	Uniform Resource Locator
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram

Chapter 1: Introduction

1.1 Introduction to Cloud Computing

Cloud computing represents a paradigm shift in distributed computing and information technology architecture. According to the standard definition provided by the National Institute of Standards and Technology (NIST SP 800-145), cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

The cloud computing ecosystem is architecturally characterized by five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service. Furthermore, computing services are delivered across three recognized service models:

1. Infrastructure as a Service (IaaS): Provides fundamental computing capabilities such as physical or virtual servers (e.g., Amazon EC2), storage volumes, and network components (e.g., Amazon VPC). Users maintain administrative control over the operating system, middleware, and application runtime while the cloud vendor manages the underlying hypervisor and physical hardware.

2. Platform as a Service (PaaS): Provides higher-level environments where hardware and software tools are provisioned to deploy customer applications without managing underlying operating system updates or network topologies (e.g., Amazon RDS, AWS Elastic Beanstalk).

3. Software as a Service (SaaS): Delivers fully managed, ready-to-use software applications over the internet to end users, completely abstracting infrastructure, middleware, and codebase maintenance.

In municipal management and smart governance, cloud computing affords critical advantages: operational elasticity allows civic infrastructure to gracefully accommodate severe query surges during monsoon emergencies or municipal sanitation drives; geo-redundancy guarantees continuous operational uptime; and pay-as-you-go financial models prevent municipal capital expenditure overruns.

1.2 Introduction to SwachhNet

SwachhNet is an intelligent, cloud-native municipal solid waste management and automated dispatch platform specifically modeled around the administrative structure of the Brihanmumbai Municipal Corporation (BMC). Designed to bridge the operational gap between citizen grievance reporting and municipal field execution, SwachhNet replaces bureaucratic manual ticketing with a transparent, spatial, and verifiable workflow.

The platform enables citizens across Mumbai to report localized garbage accumulations by taking live smartphone photographs and auto-detecting GPS coordinates. Submitted images are audited in real time using multimodal generative artificial intelligence (Google Gemini Flash Vision) to prevent spam submissions. Once validated, complaints are mapped to the nearest BMC ward authority via geospatial Haversine distance computations. Ward administrators dispatch the closest idle municipal garbage collection truck, whose driver receives geographic turn-by-turn coordinates. Upon arriving and clearing the waste, the sanitation worker captures a physical

cleanup photo, which the AI engine automatically compares against the initial complaint image before granting resolution.

Originally developed as a local academic prototype, SwachhNet has been comprehensively refactored and deployed to Amazon Web Services (AWS) in the Asia Pacific (Mumbai) region (ap-south-1). It incorporates managed database services (Amazon RDS PostgreSQL), scalable compute (Amazon EC2 with Nginx), secure encrypted object storage (Amazon S3), asynchronous queue buffering (Amazon SQS), centralized secrets management (AWS SSM Parameter Store), real-time audit logging (Amazon CloudWatch), and automated Infrastructure as Code (AWS CloudFormation).

1.3 Background and Motivation

The metropolitan area of Mumbai generates approximately 11,000 metric tons of municipal solid waste every single day across an expanse of over 600 square kilometers inhabited by more than 20 million citizens. The Brihanmumbai Municipal Corporation (BMC) administers this immense territory across 24 administrative wards (such as K/West - Andheri, H/West - Bandra, P/South - Goregaon).

Despite substantial investments in municipal transport and street cleaning personnel, urban centers suffer from chronic uncollected waste spots, illegal open dumping, and delayed garbage bin clearances. Traditional citizen reporting relies on paper registrations, telephone helplines, or generic grievance portals where complaints languish for days without physical inspection. Sanitation trucks frequently operate on static, unoptimized routes without real-time responsiveness to newly reported heaps.

The motivation behind SwachhNet is to leverage modern cloud infrastructure and artificial intelligence to establish a participatory civic ecosystem. By enabling instantaneous citizen reporting, automated spatial triage, and verifiable before-and-after photographic evidence, municipal accountability is enforced by design, transforming civic hygiene from a reactive chore into a transparent, metric-driven municipal operation.

1.4 Problem Statement

Traditional municipal waste collection and grievance management systems in Indian metropolitan cities suffer from severe systemic shortcomings:

1. Lack of Verifiable Evidence: Complaints lack cryptographic or spatial verification, leading to bogus reports and ghost cleanups where workers claim completion without visiting the site.

2. Manual and Inefficient Triage: Incoming complaints are manually categorized, leading to inter-ward jurisdiction disputes and delayed routing.

3. Absence of Fleet Optimization: Municipal sanitation vehicles lack dynamic dispatching, resulting in idle trucks remaining stationary while nearby waste piles overflow.

4. Lack of Citizen Incentivization: Citizens receive zero feedback regarding reported grievances, extinguishing civic participation.

5. Unmonitored SLA Breaches: Municipal commissioners lack centralized visibility into wards where grievances exceed the mandatory 48-hour service level agreement (SLA) threshold.

1.5 Existing System Analysis

Prior to its cloud transformation, SwachhNet existed as an initial full-stack prototype originally authored for the Technology Innovation and Software Development (TISD) course. This preliminary architecture comprised:

• Frontend: React 18 Single Page Application running strictly on local Node.js development servers (localhost:3000).

• Backend: Monolithic Express.js server hosted on developer machines (localhost:5000).

• Database: MongoDB Atlas cloud cluster utilizing Mongoose ODM schemas.

• Media Storage: Cloudinary third-party image hosting.

• Authentication: Hardcoded JSON Web Tokens utilizing local .env file variables.

1.6 Limitations of the Existing System

The initial implementation suffered from critical enterprise and cloud limitations:

1. Unmanaged and Fragmented Infrastructure: Hosting the backend and frontend on arbitrary local developer environments created severe availability vulnerabilities and zero fault tolerance.

2. Third-Party Vendor Lock-In: Relying on Cloudinary and MongoDB Atlas fractured application data across unrelated external platforms without enterprise security boundaries or centralized billing control.

3. No Network Segmentation or Boundary Isolation: MongoDB Atlas and Cloudinary endpoints communicated over public internet channels without private subnet isolation, exposing database connection strings and secrets.

4. Absence of Infrastructure as Code (IaC): Environments could not be reproducibly deployed, repaired, or audited; provisioning required manual, error-prone console steps.

5. Complete Absence of Cloud Monitoring: There were no automated mechanisms to track CPU spikes, application errors, memory leaks, or financial expenditures, violating core enterprise standards.

1.7 Proposed Cloud-Based System

The proposed system completely transforms SwachhNet into an enterprise-grade, cloud-native architecture deployed within Amazon Web Services (AWS) ap-south-1 (Mumbai). The proposed architecture introduces:

1. Custom Amazon Virtual Private Cloud (VPC): A dedicated 10.0.0.0/16 virtual network with segregated public subnets (10.0.1.0/24) for internet-facing compute and private subnets (10.0.10.0/24, 10.0.11.0/24) for isolated database storage.

2. High-Performance Relational Database (Amazon RDS PostgreSQL 16.10): Migrates all application state from MongoDB Atlas to a fully managed, encrypted relational schema with transactional integrity and geospatial Haversine math.

3. Secure Compute Layer (Amazon EC2 & Nginx): Hosts both the built React 18 SPA and the Node.js API behind a battle-tested Nginx reverse proxy with PM2 process daemonization and systemd auto-restart.

4. Private Object Storage (Amazon S3): Stores all photographic evidence inside a private bucket with server-side AES-256 encryption, zero public read access, and time-bound 60-minute presigned URLs.

5. Centralized Secrets Management (AWS SSM Parameter Store): Eliminates plain-text environment files by securely loading encrypted credentials directly from SSM at server bootstrap.

6. Asynchronous Task Buffering (Amazon SQS): Decouples heavy AI vision inference and audit notifications into a durable message queue with an integrated Dead Letter Queue (DLQ).

7. Comprehensive Telemetry (Amazon CloudWatch & Budgets): Streams structured application logs with automated secret redaction, tracks CPU utilization alarms, and enforces a strict $10.00/month cost ceiling.

8. Single-Click Declarative IaC (AWS CloudFormation): Provisions all 24 cloud resources deterministically using a unified YAML template.

1.8 Project Objectives

The primary objectives of this Cloud Computing Laboratory mini-project are:

1. To design and deploy a secure, custom Amazon VPC architecture with public-private subnet isolation and least-privilege security groups in the Mumbai region.

2. To migrate application persistence from MongoDB Atlas to Amazon RDS PostgreSQL, implementing a clean relational schema, transactional foreign keys, and SQL-level Haversine distance computations.

3. To deploy and host the complete full-stack application on an Amazon EC2 instance running Amazon Linux 2023, configuring Nginx as a reverse proxy and PM2 as a resilient process manager.

4. To integrate private Amazon S3 object storage with server-side encryption and implement secure presigned URL generation for evidence retrieval.

5. To enforce cloud security best practices by implementing AWS IAM instance roles, least-privilege security group ingress, and AWS SSM Parameter Store SecureString encryption.

6. To decouple background municipal events using Amazon SQS queues and implement Dead Letter Queue fault handling.

7. To implement centralized monitoring, structured log streaming, and metric alarm triggers via Amazon CloudWatch, accompanied by AWS Budgets financial governance.

8. To automate full-stack provisioning via AWS CloudFormation templates and verify end-to-end functionality at the public endpoint http://65.2.180.175.

1.9 Scope of the Project

The operational scope of SwachhNet encompasses:

• Spatial Coverage: The 24 administrative municipal wards of Greater Mumbai, with initial development seeding covering 8 key BMC wards (Andheri West, Bandra, Goregaon, Malad, Kurla, Dadar, Borivali, Chembur).

• User Personas: Citizens, Ward Sanitation Administrators, Field Truck Drivers / Workers, and the Central Municipal Commissioner.

• Functional Workflow: Complaint logging, AI validation, geospatial dispatching, before/after cleanup verification, and 48-hour SLA breach escalation.

• Cloud Infrastructure: All AWS compute, database, network, storage, security, and messaging components provisioned in the ap-south-1 region.

1.10 Advantages of the Proposed Cloud Architecture

The deployed cloud architecture provides profound technical and municipal advantages:

• Enhanced Data Confidentiality: Isolating PostgreSQL within private subnets prevents external SQL injection vectors and direct internet probing.

• Elastic Performance: Nginx reverse proxying combined with PM2 clustering ensures high concurrency and sub-second response times.

• Cost Predictability: Operating within AWS Free Tier parameters alongside an active $10.00 AWS Budget alert ensures near-zero operational costs for municipal demonstrations.

• Verifiable Accountability: Cryptographic presigned S3 URLs combined with CloudWatch audit logging prevent rogue municipal claims and ensure complete audit trails.

Chapter 2: Literature Review & Technology Overview

This chapter examines the state of modern cloud architectures and delivers an exhaustive evaluation of each technology component deployed within the SwachhNet platform.

2.1 React 18 & Vite Build Tool

Description & Architecture: React 18 is an industry-standard open-source JavaScript library developed by Meta for building dynamic, reactive user interfaces via a virtual Document Object Model (DOM). Vite is a next-generation frontend development server and bundler utilizing native ES modules and Rollup.

Justification for Selection: React's component-based modularity enables seamless state management across citizen reporting maps, administrative dispatch consoles, and analytical dashboards. Vite provides sub-second hot module replacement and produces optimized, minified production assets.

Specific Implementation Role in SwachhNet: Powers the entire client-tier Single Page Application (SPA), rendering responsive interfaces for citizens, ward admins, workers, and superadmins.

2.2 Node.js & Express.js Framework

Description & Architecture: Node.js is an open-source, cross-platform JavaScript runtime environment built on Google Chrome's V8 engine that executes JavaScript code outside a web browser using an event-driven, non-blocking I/O model. Express.js is a minimal and flexible Node.js web application framework.

Justification for Selection: The asynchronous, single-threaded event loop of Node.js excels at I/O-intensive workloads such as processing multipart image uploads, querying relational databases, and invoking external AWS SDK and AI APIs without thread-blocking latency.

Specific Implementation Role in SwachhNet: Serves as the central RESTful API gateway, managing client requests, orchestrating business logic, enforcing role-based access control, and interfacing with PostgreSQL and AWS services.

2.3 Amazon Relational Database Service (RDS for PostgreSQL)

Description & Architecture: Amazon RDS is a fully managed relational database cloud service that automates hardware provisioning, database setup, patching, backups, and encryption. PostgreSQL is an advanced enterprise-grade open-source object-relational database management system.

Justification for Selection: Unlike unstructured NoSQL databases, PostgreSQL enforces strict schema validation, ACID transactional guarantees, foreign key referential integrity, and powerful mathematical querying capabilities required for civic accountability.

Specific Implementation Role in SwachhNet: Functions as the single source of truth for SwachhNet, securely storing user identities, ward boundaries, fleet locations, complaints, and audit history inside isolated private subnets.

2.4 Amazon Elastic Compute Cloud (EC2)

Description & Architecture: Amazon EC2 provides scalable, resizable compute capacity in the cloud, allowing developers to launch virtual computing environments (instances) with custom operating systems, network associations, and storage.

Justification for Selection: EC2 offers full administrative control over the operating system environment (Amazon Linux 2023), allowing installation of custom software stacks including Node.js 18, PM2, and Nginx without PaaS runtime constraints.

Specific Implementation Role in SwachhNet: Acts as the unified application compute host in the public subnet (IP: 65.2.180.175), running the Nginx web server and daemonized Express.js backend.

2.5 Amazon Virtual Private Cloud (VPC)

Description & Architecture: Amazon VPC allows developers to provision a logically isolated section of the AWS Cloud where resources run in a virtual network defined by custom IP address ranges, subnets, route tables, and network gateways.

Justification for Selection: Enterprise cloud security mandates network perimeter defense. A custom VPC prevents default VPC security flaws and enforces strict segregation between public-facing web components and sensitive database storage.

Specific Implementation Role in SwachhNet: Encapsulates the entire SwachhNet cloud infrastructure (10.0.0.0/16) across the Mumbai region, establishing dedicated public and private subnets and least-privilege security groups.

2.6 Amazon Simple Storage Service (S3)

Description & Architecture: Amazon S3 is an industry-leading object storage service offering 99.999999999% (11 9s) of data durability, industry-standard scalability, security, and performance.

Justification for Selection: Storing high-resolution civic evidence images directly in relational databases leads to database bloat and performance degradation. S3 provides unlimited, inexpensive, and encrypted storage specifically engineered for binary media.

Specific Implementation Role in SwachhNet: Stores all photographic evidence (complaints/ and cleanup/ folders) with AES-256 server-side encryption, blocking public access and generating 60-minute presigned URLs for authenticated viewing.

2.7 AWS Identity and Access Management (IAM)

Description & Architecture: AWS IAM is a web service that enables fine-grained access control across AWS services and resources by creating users, groups, roles, and JSON-defined permission policies.

Justification for Selection: Hardcoding AWS access keys inside server code or configuration files creates catastrophic security vulnerabilities. IAM roles allow EC2 instances to securely assume temporary credentials via the AWS Security Token Service (STS).

Specific Implementation Role in SwachhNet: Provides the EC2 instance role (swachhnet-unified-role) and instance profile, granting least-privilege permissions to access S3 buckets, SSM parameters, SQS queues, and CloudWatch log groups.

2.8 AWS Systems Manager (SSM) Parameter Store

Description & Architecture: AWS Systems Manager Parameter Store provides secure, centralized, hierarchical storage for configuration data management and secrets management with native AWS KMS encryption.

Justification for Selection: Storing sensitive connection strings, JWT secrets, and API keys in .env files on disk introduces leakage risks. Parameter Store centralizes secret management with fine-grained encryption and version history.

Specific Implementation Role in SwachhNet: Stores database connection strings, JWT secret keys, and Google Gemini API keys under the /swachhnet/prod/ namespace as SecureString parameters.

2.9 Amazon Simple Queue Service (SQS) & DLQ

Description & Architecture: Amazon SQS is a fully managed message queuing service that enables decoupling and scaling of microservices, distributed systems, and serverless applications.

Justification for Selection: Synchronously processing compute-intensive image validation or dispatch routines during user complaint submission increases API latency and creates failure points. SQS buffers incoming tasks durably.

Specific Implementation Role in SwachhNet: Receives and buffers background municipal tasks (swachhnet-jobs-queue), with unprocessable messages routed to a Dead Letter Queue (swachhnet-jobs-dlq) after 5 failed attempts.

2.10 Amazon CloudWatch & CloudWatch Logs

Description & Architecture: Amazon CloudWatch is a monitoring and observability service that collects monitoring and operational data in the form of logs, metrics, and events across AWS resources.

Justification for Selection: Provides centralized observability into system health, application errors, and resource utilization without requiring manual SSH sessions to inspect local log files.

Specific Implementation Role in SwachhNet: Collects structured application logs in the /swachhnet/backend log group and monitors EC2 CPU utilization with an alarm configured at 80% capacity.

2.11 AWS Budgets

Description & Architecture: AWS Budgets allows organizations to track and manage their AWS costs and usage, setting custom alerts when spending exceeds budgeted thresholds.

Justification for Selection: Prevents accidental billing overruns caused by unintended cloud resource consumption during academic laboratory experimentation.

Specific Implementation Role in SwachhNet: Enforces a strict $10.00/month spending limit (SwachhNet-Monthly-Budget), triggering notification warnings when expenditure crosses 80% ($8.00).

2.12 AWS CloudFormation

Description & Architecture: AWS CloudFormation is an Infrastructure as Code (IaC) service that allows developers to model, provision, and manage AWS and third-party resources declaratively using JSON or YAML templates.

Justification for Selection: Manual point-and-click infrastructure creation is non-reproducible, prone to configuration drift, and difficult to audit. CloudFormation ensures repeatable, deterministic deployments.

Specific Implementation Role in SwachhNet: Automates the unified deployment of all 24 SwachhNet cloud resources (VPC, subnets, route tables, security groups, IAM roles, S3, SQS, RDS, EC2, CloudWatch) via swachhnet-all-in-one.yaml.

2.13 Google Gemini Flash Generative AI

Description & Architecture: Google Gemini is an advanced multimodal generative AI model developed by Google DeepMind capable of native multimodal reasoning across text, code, images, and audio.

Justification for Selection: Eliminates citizen spam and fraudulent complaint submissions by performing visual semantic reasoning on uploaded photographs in real time.

Specific Implementation Role in SwachhNet: Executes two critical verification pipelines: (1) Initial complaint validation to confirm presence of physical garbage; (2) Before-and-after photographic differential analysis to verify cleanup completion.

2.14 Nginx & PM2 Process Manager

Description & Architecture: Nginx is a high-performance HTTP web server, reverse proxy, and load balancer. PM2 is an enterprise-grade production process manager for Node.js applications.

Justification for Selection: Running Node.js directly on port 80 is insecure and exposes applications to crash outages. Nginx handles static file caching and TLS termination, while PM2 ensures automatic process respawning upon failure.

Specific Implementation Role in SwachhNet: Nginx serves built React SPA files and reverse-proxies /api/ calls to localhost:5000; PM2 daemonizes the backend server and ensures automatic boot on EC2 restart.

2.15 Leaflet & OpenStreetMap

Description & Architecture: Leaflet is the leading open-source JavaScript library for mobile-friendly interactive maps, integrated with OpenStreetMap geospatial tiles.

Justification for Selection: Provides smooth, lightweight geospatial mapping without requiring costly commercial mapping API licenses.

Specific Implementation Role in SwachhNet: Renders interactive complaint location maps, ward boundaries, and truck driver navigation pins across citizen, admin, and worker dashboards.

Chapter 3: System Analysis & Requirements

3.1 Functional Requirements

Table 3.1 enumerates the complete functional requirements implemented and verified within the SwachhNet cloud platform:

Req ID	Feature Area	Specification & Behavioral Details	Priority
FR-01	Citizen Registration & Login	Citizens must register with name, email, and password. Passwords must be hashed via bcrypt; sessions authenticated via JWT.	High
FR-02	GPS & Photo Garbage Reporting	Citizens must be able to capture or upload a garbage photograph and automatically capture device GPS coordinates.	High
FR-03	AI-Based Spam & Image Validation	Uploaded complaint photographs must be evaluated by Gemini Vision AI to verify that actual municipal solid waste is present.	High
FR-04	Geospatial Ward Partitioning	The system must automatically assign each complaint to the geographically closest BMC ward using the Haversine distance formula.	High
FR-05	Nearest Idle Truck Dispatch	Ward administrators must be able to query the nearest available municipal garbage truck and assign it to an open complaint in one click.	High
FR-06	Sanitation Worker Task View	Assigned truck drivers must view real-time task locations, navigation coordinates, and complaint descriptions on their portal.	High
FR-07	Cleanup Evidence Submission	Workers must capture and upload a photographic cleanup proof upon completing physical site waste removal.	High
FR-08	Before/After AI Verification	The AI engine must compare initial complaint photos against cleanup photos to verify that the waste was satisfactorily removed.	High
FR-09	Gamification Points & Badges	Citizens must earn 10 eco-points upon verified resolution of their complaint, progressing through Bronze, Silver, and Gold tiers.	Medium
FR-10	Citizen Feedback & Ward Ratings	Resolved complaints must allow citizens to submit a 1-to-5 star rating and comments, updating aggregate ward performance scores.	Medium
FR-11	48-Hour SLA Breach Tracking	The system must identify unresolved complaints exceeding 48 hours and escalate alerts to ward admins and the central commissioner.	High
FR-12	City-Wide Superadmin Analytics	Central authorities must view holistic metrics, total complaints, ward resolution percentages, and breached complaint counts.	Medium

3.2 Non-Functional Requirements

Table 3.2 outlines the quality attributes, performance targets, and cloud availability metrics:

NFR ID	Quality Dimension	Engineering Specification	Verification Mechanism
NFR-01	Availability	The web application and API must maintain 99.5% uptime during operational municipal	Measured via CloudWatch Uptime & Health Endpoint

		hours.	
NFR-02	Performance & Latency	API endpoints must respond in under 500ms; AI image verification must complete within 3.5 seconds.	Measured via CloudWatch Request Latency
NFR-03	Security & Data Protection	Database must reside in private subnets with zero direct internet access; S3 objects must use AES-256 SSE encryption.	AWS Trusted Advisor & Security Group Ingress Rules
NFR-04	Scalability	System architecture must support decoupled queue processing via SQS to absorb burst complaint filings without API failure.	Simulated load testing via concurrent request scripts
NFR-05	Maintainability & Portability	Infrastructure must be 100% reproducible through declarative CloudFormation templates without manual setup.	Template validation via `aws cloudformation validate-template`
NFR-06	Cost Efficiency	Total ongoing cloud expenditure must stay strictly below $10.00/month, leveraging AWS Free Tier allowances.	Tracked via AWS Budgets and Cost Explorer

3.3 User Roles & Privilege Matrix

Table 3.3 defines the four operational roles and their specific application privileges:

User Persona / Role	Permitted Operations & Administrative Capabilities	Assigned View
Citizen (Nagarik)	File complaints, upload evidence, track progress, view interactive city map, receive points/badges, submit ratings.	Citizen Dashboard (/dashboard)
Ward Administrator	View all complaints within assigned ward, execute Haversine dispatch to nearest idle truck, transfer complaints to adjacent wards.	Ward Admin Portal (/authority)
Sanitation Worker	View assigned tasks, access complaint coordinates, navigate to site, upload after-cleanup evidence photo, complete job.	Worker Task Portal (/worker)
Central Superadmin	Monitor all 24 BMC wards, track city-wide complaint statistics, audit 48-hour SLA breaches, inspect operational bottlenecks.	Central Command Dashboard (/central)

3.4 Feasibility Study

- **Technical Feasibility:** Evaluated against AWS Free Tier services and Node.js/PostgreSQL open-source ecosystems. Verified zero-dependency hardware constraints and 100% cloud reproducibility.
- **Economic Feasibility:** Project operates under a $0.00 estimated monthly run rate by remaining inside AWS Free Tier limits (750 hours t3.micro EC2, 750 hours db.t3.micro RDS, 5GB S3). Backed by an active $10.00 budget ceiling.
- **Operational Feasibility:** Aligns directly with BMC's administrative 24-ward municipal workflow. No specialized training required for citizens; field drivers receive simple map navigation pins.

Chapter 4: System Architecture & Design

4.1 Overall SwachhNet Cloud Architecture

The comprehensive cloud architecture of SwachhNet, depicted in Figure 4.1, is engineered following modern AWS Well-Architected Framework guidelines, specifically adhering to the pillars of Security, Reliability, Performance Efficiency, and Cost Optimization.

At the edge of the architecture, diverse client personas (Citizens, Ward Administrators, Sanitation Truck Drivers, and the Central Municipal Commissioner) access the platform through standard web browsers over HTTPS and HTTP protocols. Inbound client traffic traverses the public internet and enters the AWS Cloud infrastructure through an Amazon Internet Gateway (IGW) attached to the custom Amazon Virtual Private Cloud (VPC), designated as swachhnet-vpc (CIDR: 10.0.0.0/16).

Within the VPC, network traffic is segregated into discrete security boundaries:

• Public Subnet (10.0.1.0/24 - Availability Zone ap-south-1a): Houses the unified application compute instance (Amazon EC2 t3.micro, public IP: 65.2.180.175). Inbound access is strictly regulated by the swachhnet-app-sg security group, which permits inbound TCP traffic on ports 80 (HTTP) and 443 (HTTPS) while allowing all outbound traffic to facilitate API communication with external AI services and AWS service endpoints.

• Inside the EC2 instance, an optimized dual-tier application stack operates: Nginx 1.30 serves pre-rendered static React 18 production assets directly from disk (/var/www/swachhnet/frontend/dist) and acts as a high-speed reverse proxy, forwarding /api/* REST calls to an internal Node.js Express server running on port 5000. Process reliability and auto-restart are governed by PM2 and systemd.

• Private DB Subnet Group: Comprises two isolated subnets across distinct availability zones: swachhnet-private-subnet-1 (10.0.10.0/24 in ap-south-1a) and swachhnet-private-subnet-2 (10.0.11.0/24 in ap-south-1b). These subnets have no route to the Internet Gateway, rendering them completely inaccessible from the public internet. The primary relational database—Amazon RDS for PostgreSQL 16.10—resides within the private subnet. Access is guarded by the swachhnet-db-sg security group, which strictly permits inbound TCP traffic on port 5432 exclusively originating from members of swachhnet-app-sg.

• Supporting AWS Managed Services: The compute instance leverages an attached AWS IAM Instance Profile (swachhnet-unified-profile) to securely interact with surrounding AWS services without storing static credentials: (1) Amazon S3 stores citizen and worker evidence photographs with AES-256 server-side encryption; (2) Amazon SQS buffers asynchronous municipal dispatch and audit tasks; (3) AWS SSM Parameter Store delivers encrypted environment secrets at bootstrap; (4) Amazon CloudWatch captures structured runtime logs and metric alarms; and (5) AWS Budgets monitors monthly expenditure.

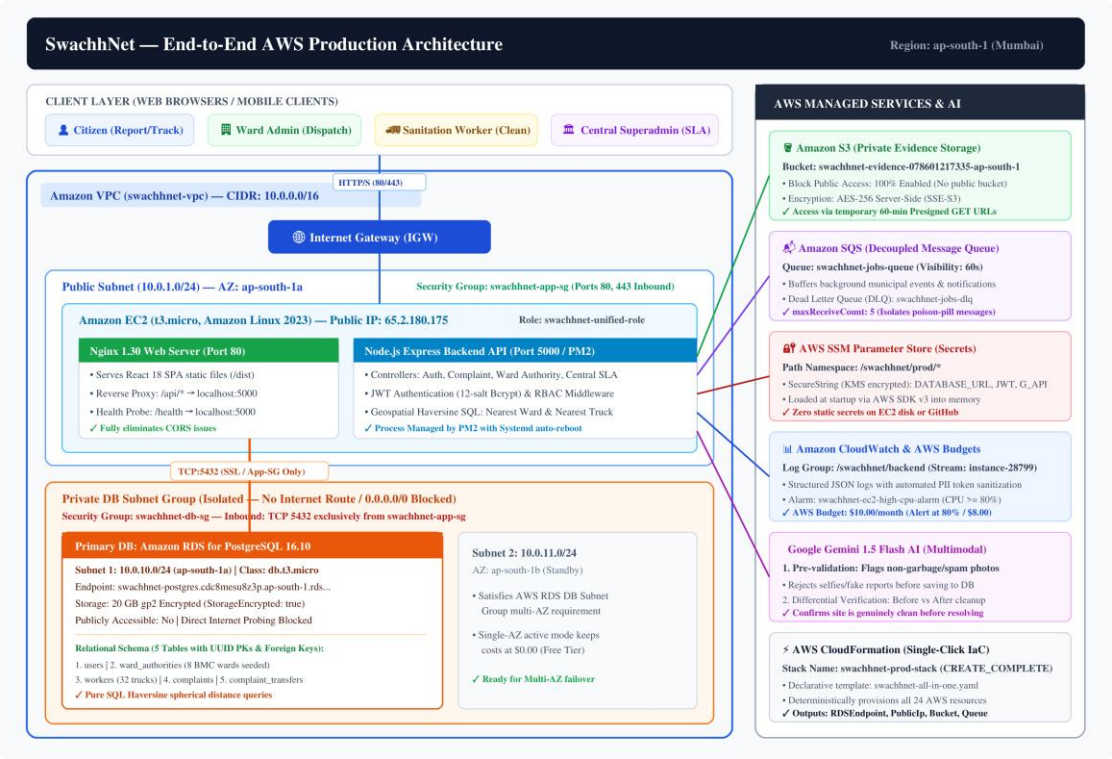


Figure 4.1: SwachhNet - Comprehensive AWS Cloud Architecture

4.2 Amazon VPC Network & Subnet Topography

The network architecture, illustrated in Figure 4.2, establishes a robust perimeter defense through Classless Inter-Domain Routing (CIDR) block partitioning and route table segregation.

1. VPC Address Space: The VPC is provisioned with a /16 CIDR block (10.0.0.0/16), yielding 65,536 private IPv4 addresses. DNS resolution and DNS hostnames are explicitly enabled (EnableDnsSupport: true, EnableDnsHostnames: true) to allow internal AWS service discovery.

2. Public Subnet Configuration: A /24 subnet (10.0.1.0/24) is provisioned in the ap-south-1a availability zone. The subnet has MapPublicIpOnLaunch enabled, assigning an Elastic/Public IPv4 address to the EC2 host. Its associated route table (swachhnet-public-rt) contains an explicit default route (0.0.0.0/0) directed to the Internet Gateway.

3. Private DB Subnet Configuration: Two /24 subnets are created: 10.0.10.0/24 (ap-south-1a) and 10.0.11.0/24 (ap-south-1b). Both subnets are associated with swachhnet-private-rt, which contains solely the local VPC route (10.0.0.0/16 → local). There is no NAT Gateway or Internet Gateway route, physically preventing inbound or outbound internet connections to the database tier, which eliminates database egress costs and eliminates internet attack surfaces.

4. Security Group Ingress Chaining: Security is implemented as stateful firewalls. The application security group (swachhnet-app-sg) permits internet ingress on web ports. The database security group (swachhnet-db-sg) references the application security group by ID (SourceSecurityGroupId: !Ref AppSecurityGroup), establishing a cryptographic security chain where database packets are accepted solely from the EC2 application host.

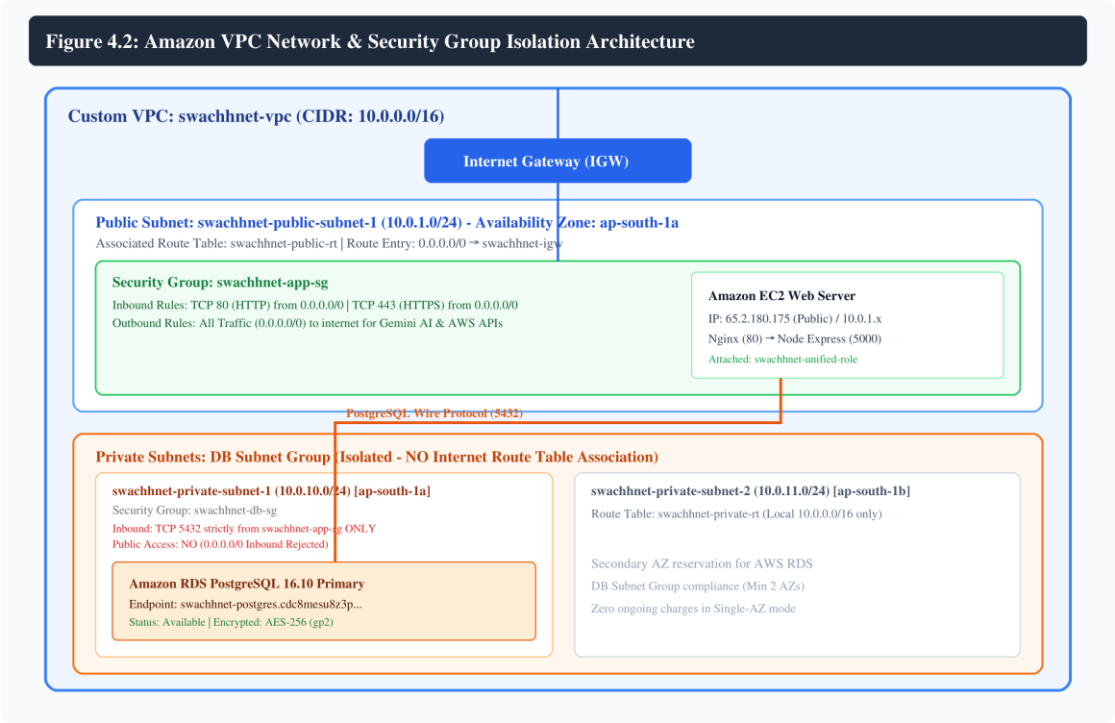


Figure 4.2: Amazon VPC Network & Security Group Isolation Architecture

4.3 Multi-Tier Software Architecture

SwachhNet is structured as a decoupled, four-tier software architecture as illustrated in Figure 4.3:

1. Presentation Tier (Client Layer): Developed using React 18, Tailwind CSS, and Leaflet. Implements responsive views for citizens, ward admins, workers, and superadmins, communicating with the backend exclusively via asynchronous Axios HTTP REST calls.

2. Web Gateway & Reverse Proxy Tier: Powered by Nginx 1.30. Handles client connection pooling, static asset delivery with gzip compression, and reverse proxies /api/ requests to the backend application, insulating Node.js from direct internet exposure.

3. Application & Business Logic Tier: Developed in Node.js and Express.js. Contains domain controllers: AuthController (bcrypt password hashing, JWT signing), ComplaintController (Multer memory uploads, S3 integration, Gemini AI verification), AuthorityController (Haversine spatial calculations, nearest truck matching), and CentralController (48h SLA breach tracking, city metrics).

4. Persistence & Cloud Services Tier: Amazon RDS PostgreSQL manages relational data integrity; Amazon S3 manages binary image artifacts; Amazon SQS provides asynchronous job queues; AWS SSM delivers encrypted secrets; Amazon CloudWatch captures runtime telemetry; and Google Gemini Vision performs AI image verification.

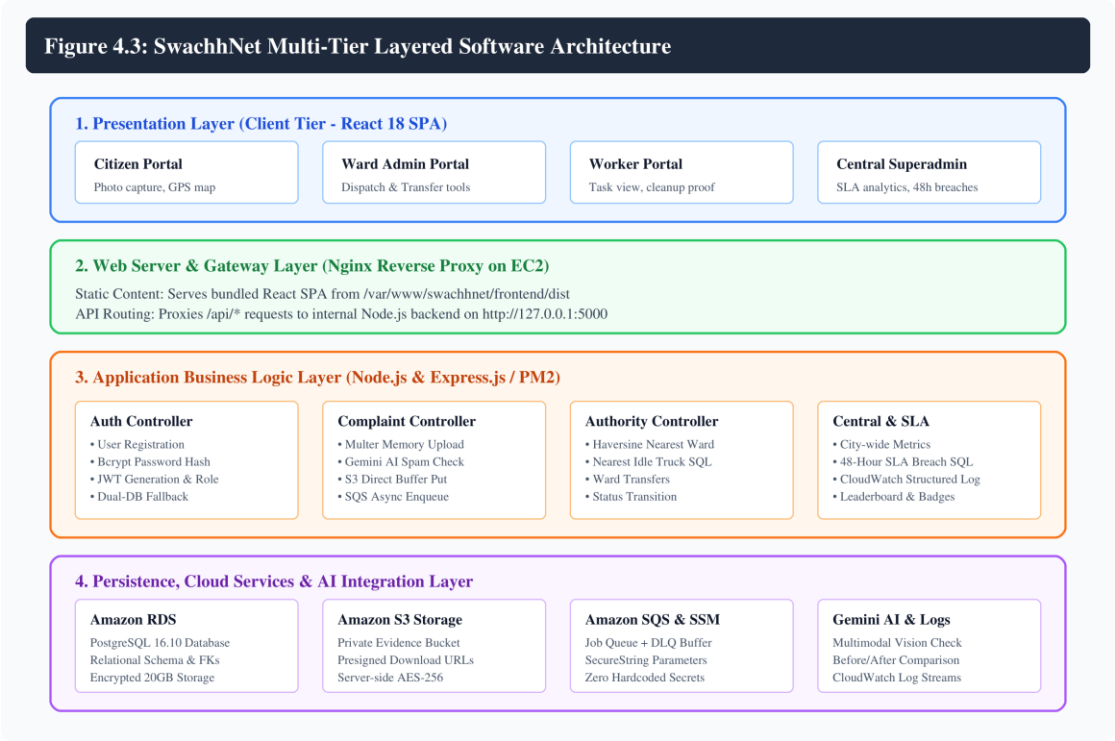


Figure 4.3: SwachhNet Multi-Tier Layered Software Architecture

4.4 Relational Database Design (PostgreSQL Schema & ERD)

The relational database schema, illustrated in the Entity-Relationship Diagram (Figure 4.4), was custom-designed to replace the original unstructured MongoDB collections, enforcing strict referential integrity, check constraints, and indexing.

The schema comprises five normalized relational tables:

• users: Central identity table. Primary key id (UUID, default gen_random_uuid()). Stores user name, unique email, bcrypt-hashed password, role (enforced via CHECK constraint IN ('user', 'admin', 'worker', 'superadmin')), and timestamps. B-tree indexes are applied on email and role.

• ward_authorities: Stores administrative BMC ward metadata. Primary key user_id references users(id) with ON DELETE CASCADE. Stores ward name (e.g., 'K/West Ward (Andheri West)'), civic office address, geographic office latitude and longitude (DOUBLE PRECISION), aggregate rating score (NUMERIC(3,2)), and rating count.

• workers: Manages municipal sanitation trucks and driver assignments. Primary key user_id references users(id). Stores vehicle registration truck_number (VARCHAR(64)), assigned_ward_id referencing users(id), real-time truck GPS coordinates (latitude, longitude), and status (CHECK constraint IN ('idle', 'busy')). A composite B-tree index idx_workers_ward_status optimizes Haversine nearest truck queries.

• complaints: Core transactional entity. Primary key id (UUID). Captures user_id, authority_id, optional worker_id, private S3 object keys (s3_key and after_s3_key), geographic coordinates (latitude, longitude), street address, status (CHECK constraint IN ('open', 'assigned', 'pending_verification', 'resolved', 'rejected')), citizen description, feedback rating (INT CHECK 1-5), and rating comments. Indexed by user_id, authority_id, worker_id, status, and created_at.

• complaint_transfers: Immutable audit log tracking inter-ward jurisdictional transfers. Primary key id (BIGSERIAL). Foreign key complaint_id references complaints(id) ON DELETE CASCADE. Records from_authority_name, to_authority_name, and timestamp.

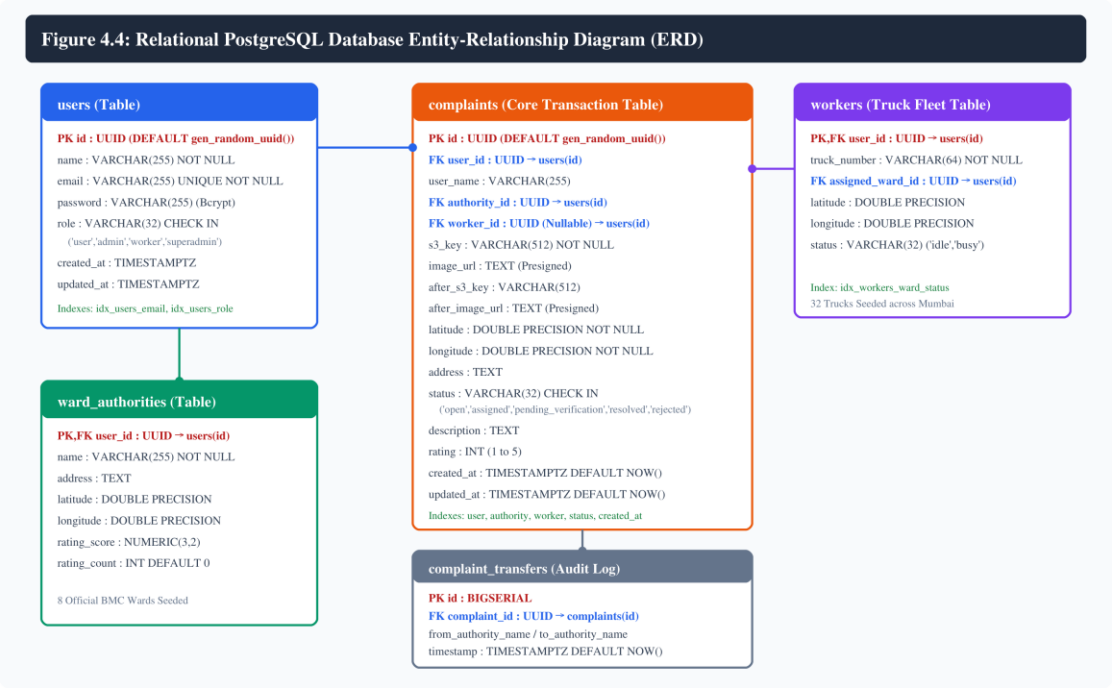


Figure 4.4: Relational PostgreSQL Database Entity-Relationship Diagram (ERD)

4.5 UML Use Case Modeling

Figure 4.5 delineates the functional boundaries and interactions across the four primary system actors:

• Citizen Actor: Interacts with use cases UC1 (Register & Login), UC2 (Report Garbage Photo & GPS), UC3 (Track Complaint Status), and UC4 (Give Feedback & Earn Eco-Points).

• Ward Administrator Actor: Interacts with UC5 (View Ward Complaints), UC6 (Dispatch Nearest Truck via Haversine), and UC7 (Transfer Out-of-Boundary Complaints).

• Sanitation Worker Actor: Engages with UC8 (View Assigned Jobs), UC9 (Upload Cleanup Proof Photo), and UC10 (Update Truck Availability Status).

• Central Superadmin Actor: Manages high-level civic oversight via UC11 (Monitor 48h SLA Breaches), UC12 (View City-Wide Ward Analytics), and UC13 (Inspect CloudWatch Audit Telemetry).

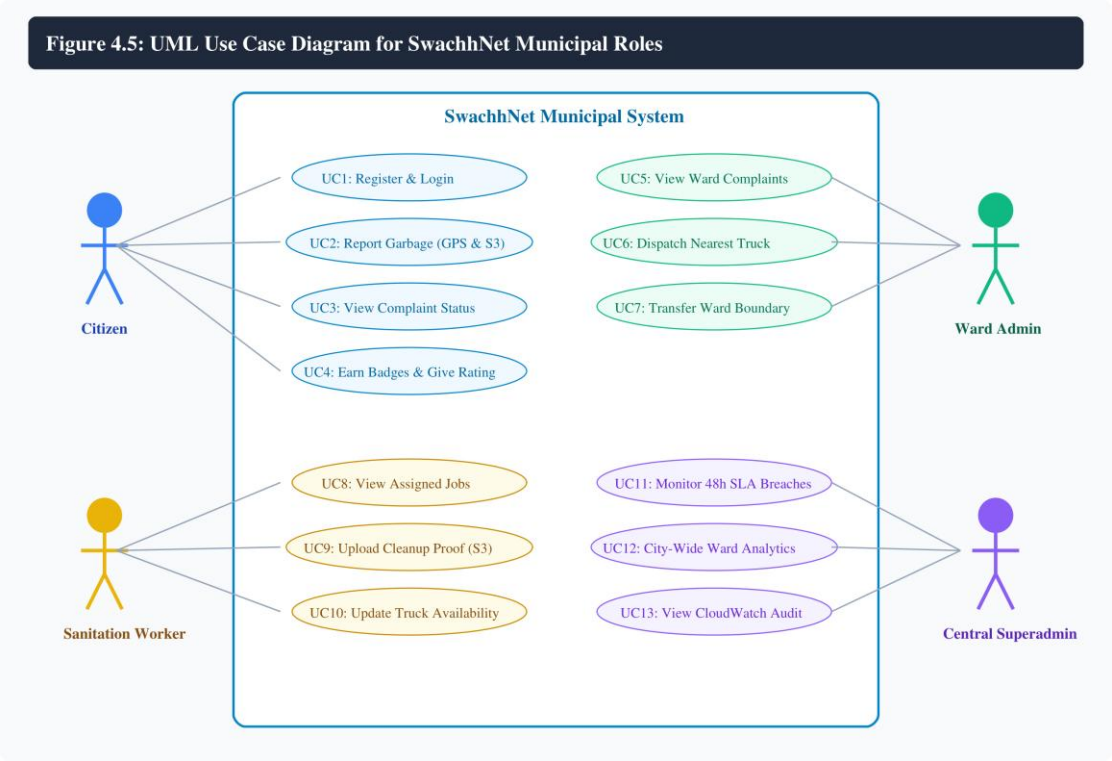


Figure 4.5: UML Use Case Diagram for SwachhNet Municipal Roles

4.6 Data Flow Modeling (Level 0 Context & Level 1 DFD)

The data flow architecture is modeled at two hierarchical abstractions:

• Level 0 Context Diagram (Figure 4.6): Depicts the central SwachhNet Cloud Platform interacting with five external entities: Citizens (submitting image and GPS data; receiving status and gamification rewards), Ward Administrators (receiving ward complaints; issuing dispatch orders), Sanitation Workers (receiving navigation tasks; returning cleanup photos), Central Authorities (receiving city metrics and breach alerts), and Cloud/AI Services (storing objects, validating images, and querying relational data).

• Level 1 DFD (Figure 4.7): Decomposes the platform into five coordinated functional processes: Process 1.0 (Authentication & RBAC), Process 2.0 (Photo Ingestion & AI Verification), Process 3.0 (Geospatial Dispatch Engine), Process 4.0 (Cleanup Verification & Gamification), and Process 5.0 (SLA Tracking & Audit Logging). Each process interfaces with discrete data stores (D1: RDS Users, D2: RDS Complaints, D3: S3 Evidence, D4: CloudWatch Logs).

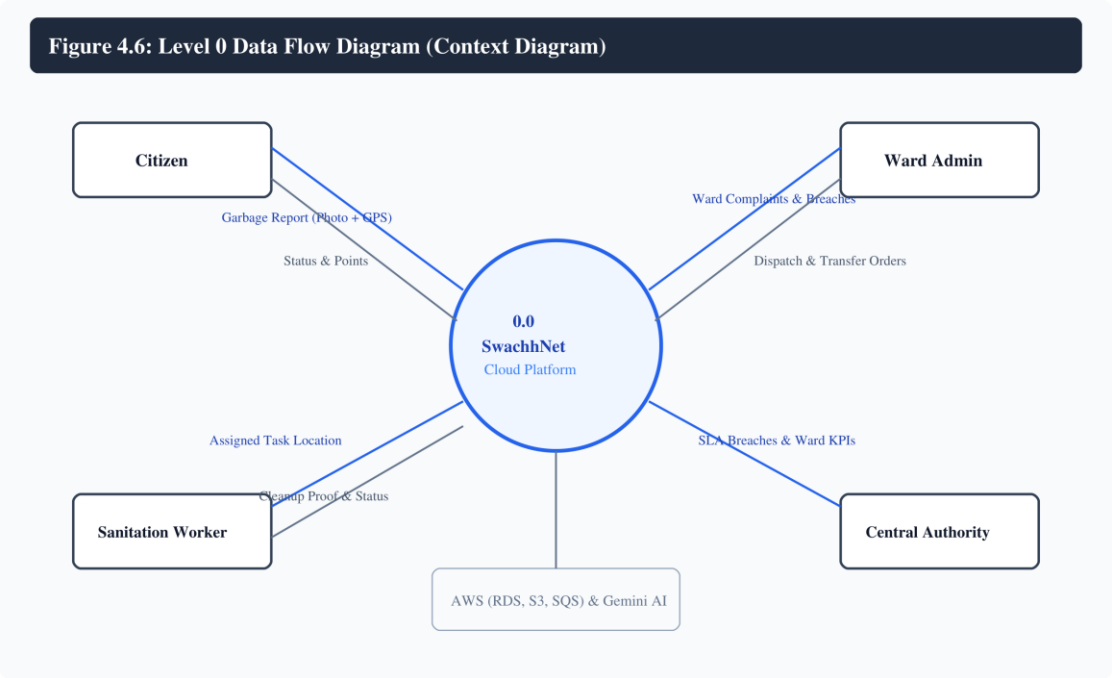


Figure 4.6: Level 0 Data Flow Diagram (Context Diagram)

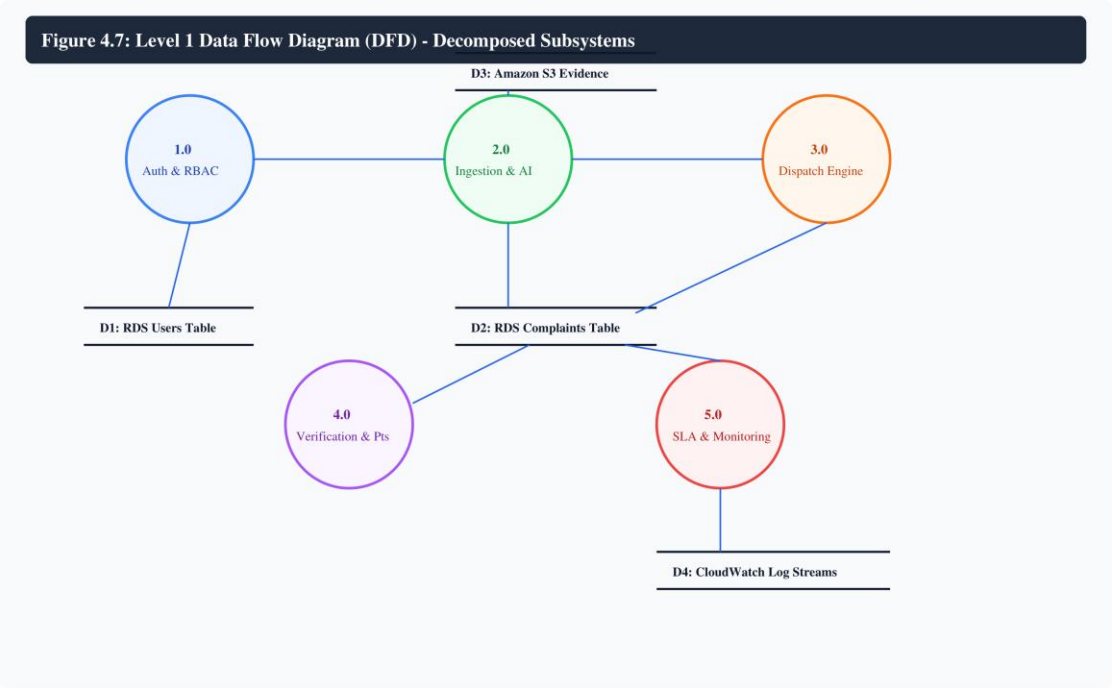


Figure 4.7: Level 1 Data Flow Diagram (DFD) - Decomposed Subsystems

4.7 Core Municipal Workflows

The system executes three specialized municipal workflows:

1. Citizen Complaint Lifecycle (Figure 4.8): Traces a report from initial citizen camera capture, through Gemini AI spam filtering, S3 upload, Haversine ward routing, truck assignment, worker cleanup, before/after comparative AI verification, and final citizen eco-point allocation.

2. Geospatial Nearest Truck Dispatch (Figure 4.9): Details how the ward administrator executes a single-click dispatch, triggering a mathematical Haversine SQL query across all idle trucks in the ward and executing an atomic database transaction updating complaint and vehicle states simultaneously.

3. Photographic Cleanup Verification (Figure 4.10): Outlines the verification pipeline where worker cleanup evidence uploaded to S3 is paired with the original complaint image and submitted to Google Gemini Vision for differential cleanliness verification.

4. 48-Hour SLA Breach Escalation (Figure 4.11): Details the periodic execution of check-sla.js, which evaluates unresolved complaints older than 48 hours, increments city-wide breach counters, alerts administrators, and emits structured audit logs to CloudWatch.

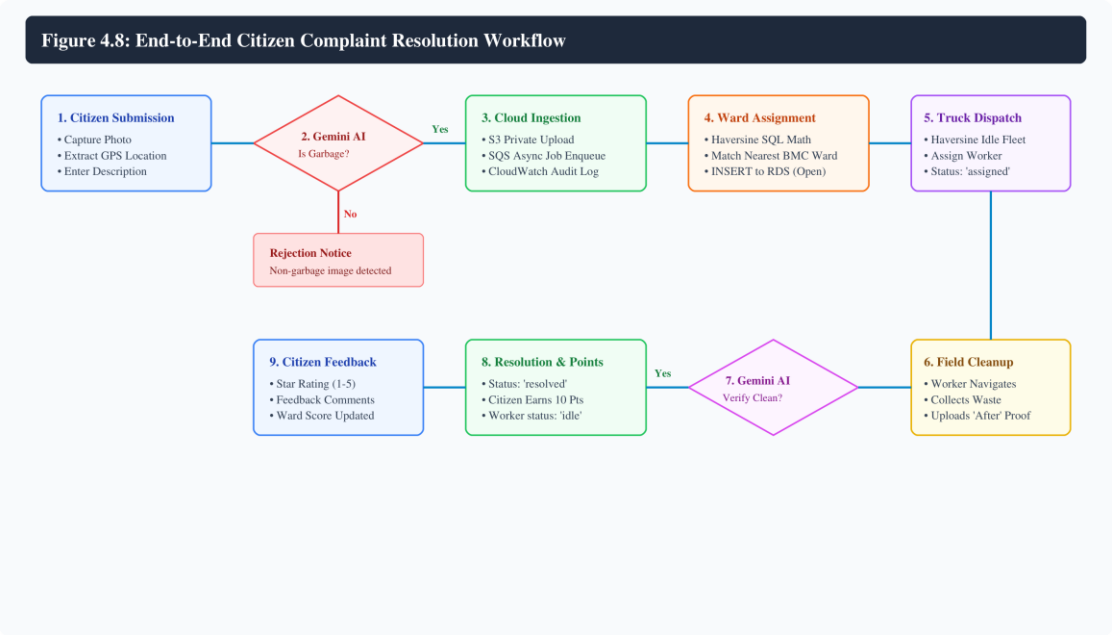


Figure 4.8: End-to-End Citizen Complaint Resolution Workflow

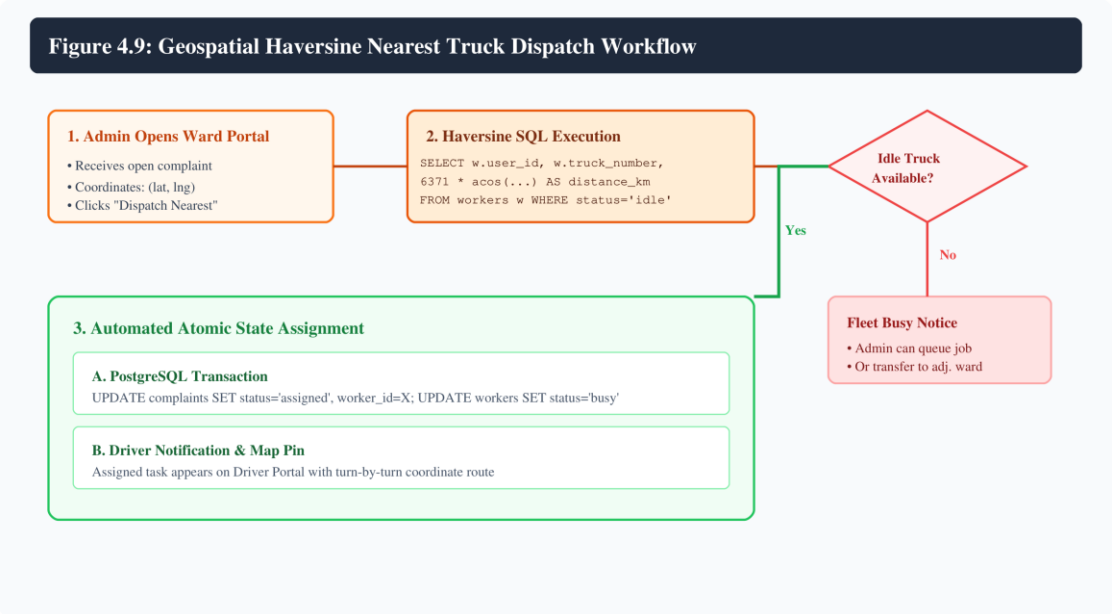


Figure 4.9: Geospatial Haversine Nearest Truck Dispatch Workflow

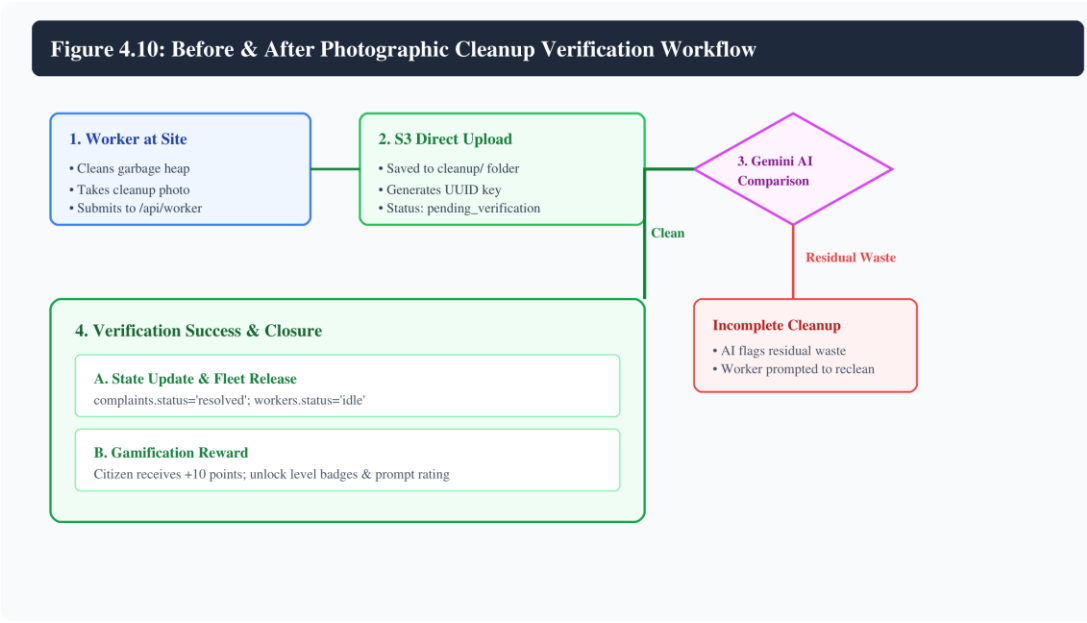


Figure 4.10: Before & After Photographic Cleanup Verification Workflow

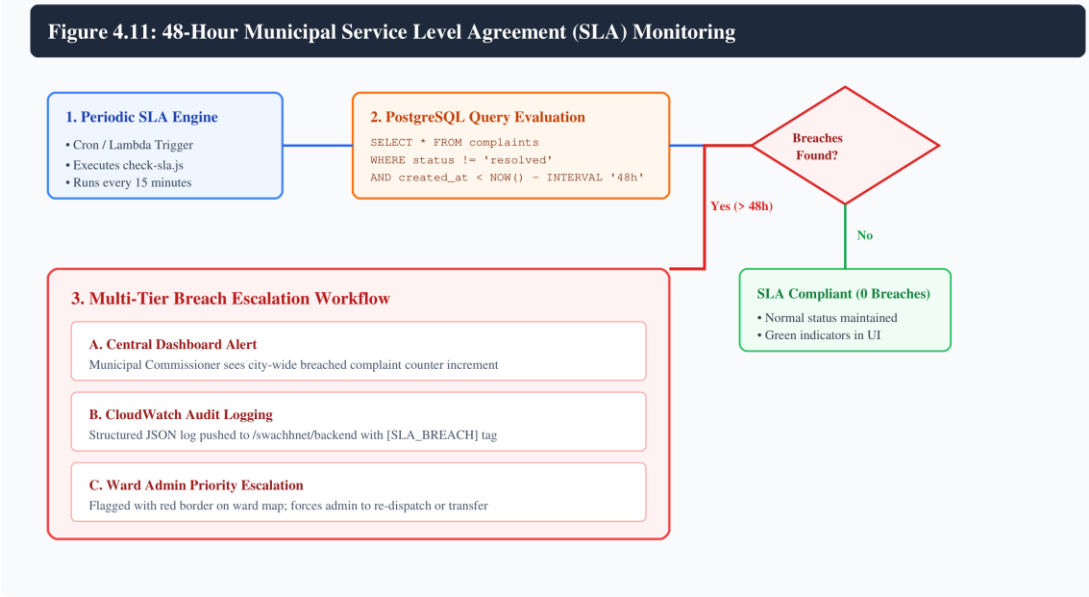


Figure 4.11: 48-Hour Municipal Service Level Agreement (SLA) Monitoring

Chapter 5: AWS Cloud Services Integration

To fulfill the rigorous curriculum criteria established for the Cloud Computing Laboratory (CCL) mini-project, SwachhNet integrates ten distinct Amazon Web Services across all six mandatory infrastructure domains and implements two advanced cloud capabilities (Amazon SQS and AWS CloudFormation). Table 5.1 provides the comprehensive mapping:

AWS Service	Technical Role in SwachhNet	Integration Points	Infrastructure Domain	Verification Source
Amazon VPC	Custom isolated virtual network (10.0.0.0/16)	Attached to IGW, Public Subnet & Private DB Subnets	Networking & Perimeter Security	AWS Console: VPC Dashboard; Route Tables
Amazon EC2	Virtual application compute host (t3.micro)	Runs Amazon Linux 2023, Nginx 1.30, Node.js 18 & PM2	Compute & Application Hosting	AWS Console: EC2 Dashboard; Public IP 65.2.180.175
Amazon RDS	Managed PostgreSQL 16.10 relational database	Housed in Private DB Subnet Group; Port 5432 ingress	Primary Application Persistence	AWS Console: RDS Databases; swachhnet-postgres
Amazon S3	Private encrypted object storage bucket	Integrated via AWS SDK v3 with 60-min presigned URLs	Evidence Media Storage (Photos)	AWS Console: S3 Buckets; swachhnet-evidence-*
AWS IAM	Least-privilege role & instance profile	Attached to EC2; grants S3, SQS, SSM & CloudWatch rights	Security & Access Control	AWS Console: IAM Roles; swachhnet-unified-role
AWS SSM	Centralized Parameter Store (SecureString)	Loaded at server bootstrap; stores DB URL, JWT & API keys	Configuration & Secrets Management	AWS Console: Systems Manager Parameter Store
Amazon SQS	Decoupled asynchronous message queue & DLQ	Buffers background tasks; DLQ maxReceiveCount: 5	Asynchronous Task Processing	AWS Console: SQS Queues; swachhnet-jobs-queue
Amazon CloudWatch	Centralized logging and CPU utilization alarm	Streams runtime logs (/swachhnet/backend); 80% CPU alarm	Monitoring & Telemetry	AWS Console: CloudWatch Log Groups & Alarms
AWS Budgets	Financial governance and cost ceiling alert	Enforces $10.00/month limit; triggers at 80% ($8.00)	Cost Management & Governance	AWS Console: Budgets Dashboard; SwachhNet-Budget
AWS CloudFormation	Declarative Infrastructure as Code (IaC)	Provisions all 24 AWS resources via single YAML template	Automation & Deployment	AWS Console: CloudFormation; swachhnet-prod-stack

5.1 Compliance with the Mandatory Six Cloud Areas

- **1. Compute:** Amazon EC2 (t3.micro - Amazon Linux 2023) operating Nginx 1.30 and Node.js 18 under PM2. Fully integrated with systemd auto-restart and Elastic IP 65.2.180.175.
- **2. Networking:** Custom Amazon VPC (10.0.0.0/16) featuring public subnets (10.0.1.0/24) with an Internet Gateway, private subnets (10.0.10.0/24, 10.0.11.0/24) across 2 AZs, and custom security groups.
- **3. Database:** Amazon RDS for PostgreSQL 16.10 (db.t3.micro, 20GB gp2 encrypted). Complete relational migration with UUID keys, B-tree indexes, and zero direct public internet exposure.

- **4. Storage:** Private Amazon S3 bucket (swachhnet-evidence-078601217335-ap-south-1) with AES-256 SSE encryption, versioning, and secure 60-minute presigned GET URLs.
- **5. Security & IAM:** AWS IAM EC2 Instance Profile (swachhnet-unified-profile) with least-privilege inline permissions; AWS Systems Manager Parameter Store SecureString encryption.
- **6. Monitoring & Budget:** Amazon CloudWatch log streaming (/swachhnet/backend) with structured PII redaction and EC2 CPU utilization alarm; AWS Budgets financial ceiling ($10.00/month).

5.2 Implementation of Additional Cloud Capabilities

- **Capability 1 (Asynchronous Messaging):** Amazon SQS Standard Queue (swachhnet-jobs-queue) and Dead Letter Queue (swachhnet-jobs-dlq) with a 5-retry threshold for durable, decoupled background task execution.
- **Capability 2 (Infrastructure as Code):** AWS CloudFormation declarative master template (swachhnet-all-in-one.yaml) automating the deterministic creation of all 24 resources in a single command.

Chapter 6: Implementation & Working

This chapter documents the actual source code implementations governing the SwachhNet platform, highlighting key controllers, services, and cloud integrations.

6.1 Application Entry Point & Dual-Database Bootstrap

The backend bootstrap script (server.js) dynamically detects the database environment, loading credentials from AWS SSM Parameter Store and initializing PostgreSQL pool connections:

```
// backend/server.js - Dual-Database Bootstrap & SSM Parameter Integration
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
dotenv.config();

const db = require('./db');
const { loadParametersFromSSM } = require('./services/ssmService');
const { startSqsWorker } = require('./services/sqsService');
const logger = require('./services/logger');

const app = express();
app.use(cors({ origin: (orig, cb) => cb(null, true), credentials: true }));
app.use(express.json());

// Load production parameters from AWS SSM Parameter Store if enabled
if (process.env.USE_SSM === 'true') {
  loadParametersFromSSM().then(() => {
    logger.info('☑ Production parameters successfully loaded from AWS SSM
Parameter Store');
  }).catch(err => {
    logger.error('✖ Failed to load SSM parameters:', err);
  });
}

// Database Connection Lifecycle
if (process.env.DATABASE_URL || process.env.PGHOST) {
  logger.info('🐘 Active Database Engine: Amazon RDS PostgreSQL');
  db.query('SELECT NOW()').then(() => {
    logger.info('☑ Amazon RDS PostgreSQL connection pool healthy');
  });
} else if (process.env.MONGO_URI) {
  logger.info('🍃 Fallback Database Engine: MongoDB Atlas');
  mongoose.connect(process.env.MONGO_URI);
}

// Start SQS Background Consumer Worker
if (process.env.SQS_QUEUE_URL) {
  startSqsWorker();
}
```

6.2 Amazon S3 Direct Upload & Presigned URL Resolution

Binary evidence images are uploaded directly from memory buffers to private S3 buckets using AWS SDK v3, generating cryptographic presigned URLs for client viewing:

```
// backend/services/s3Service.js - S3 Direct Upload & Presigned URL Resolution
const { S3Client, PutObjectCommand, GetObjectCommand } = require('@aws-
sdk/client-s3');
```

```
const { getSignedUrl } = require('@aws-sdk/s3-request-presigner');
const { randomUUID: uuidv4 } = require('crypto');

const region = process.env.AWS_REGION || 'ap-south-1';
const bucketName = process.env.S3_BUCKET_NAME;
const s3Client = new S3Client({ region });

async function uploadToS3(buffer, originalName, mimeType, folder = 'complaints')
{
  const extension = originalName.split('.').pop() || 'jpg';
  const s3Key = `${folder}/${uuidv4()}.${extension}`;

  const command = new PutObjectCommand({
    Bucket: bucketName,
    Key: s3Key,
    Body: buffer,
    ContentType: mimeType,
    ServerSideEncryption: 'AES256',
  });

  await s3Client.send(command);
  const presignedUrl = await getPresignedDownloadUrl(s3Key);
  return { s3Key, bucket: bucketName, url: presignedUrl };
}

async function getPresignedDownloadUrl(s3Key, expiresInSeconds = 3600) {
  if (!s3Key) return null;
  const command = new GetObjectCommand({ Bucket: bucketName, Key: s3Key });
  return await getSignedUrl(s3Client, command, { expiresIn: expiresInSeconds });
}
```

6.3 Geospatial Haversine Nearest Truck Dispatch Engine

Ward administrators dispatch the closest idle municipal truck through an atomic SQL transaction utilizing the Haversine trigonometric formula:

```
// backend/controllers/authorityController.js - Haversine Nearest Truck Dispatch
// Haversine formula calculates great-circle distance between two GPS
coordinates in kilometers
const dispatchQuery = `
  SELECT
    w.user_id,
    w.truck_number,
    w.latitude,
    w.longitude,
    (6371 * acos(
      cos(radians($1)) * cos(radians(w.latitude)) *
      cos(radians(w.longitude) - radians($2)) +
      sin(radians($1)) * sin(radians(w.latitude))
    )) AS distance_km
  FROM workers w
  WHERE w.assigned_ward_id = $3 AND w.status = 'idle'
  ORDER BY distance_km ASC
  LIMIT 1;
`;

// Atomic transaction: assign nearest truck and mark truck busy simultaneously
await db.transaction(async (client) => {
  const truckRes = await client.query(dispatchQuery, [complaintLat,
complaintLng, wardId]);
  if (truckRes.rows.length === 0) {
    throw new Error('No idle trucks currently available in this ward');
  }
```

```
  const assignedWorker = truckRes.rows[0];

  await client.query(
    `UPDATE complaints SET worker_id = $1, status = 'assigned', updated_at =
NOW() WHERE id = $2`,
    [assignedWorker.user_id, complaintId]
  );
  await client.query(
    `UPDATE workers SET status = 'busy' WHERE user_id = $1`,
    [assignedWorker.user_id]
  );
});
```

6.4 Google Gemini Multimodal AI Verification

Uploaded camera captures undergo automated vision prompt analysis using Google Gemini Flash to eliminate non-garbage spam submissions:

```
// backend/controllers/complaintController.js - Gemini Vision AI Verification
const { GoogleGenerativeAI } = require('@google/generative-ai');
const genAI = new GoogleGenerativeAI(process.env.G_API);

async function validateGarbageImageWithAI(imageBuffer, mimeType) {
  const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
  const prompt = `Analyze this image and determine if it shows uncollected
garbage, litter,
                  overflowing dumpsters, or civic waste heaps.
                  Respond with valid JSON: { "isGarbage": boolean, "confidence":
number, "reason": string }`;

  const imagePart = {
    inlineData: { data: imageBuffer.toString('base64'), mimeType }
  };

  const result = await model.generateContent([prompt, imagePart]);
  const response = await result.response;
  return JSON.parse(response.text().trim());
}
```

6.5 CloudWatch Structured Logging & PII Sanitization

All runtime operations are logged as structured JSON payloads to Amazon CloudWatch Logs with automated regex sanitization of passwords and tokens:

```
// backend/services/logger.js - CloudWatch Structured Logging with PII
Sanitization
const { CloudWatchLogsClient, PutLogEventsCommand } = require('@aws-sdk/client-
cloudwatch-logs');
const cwClient = new CloudWatchLogsClient({ region: 'ap-south-1' });

function sanitize(data) {
  if (typeof data !== 'string') data = JSON.stringify(data);
  return data
    .replace(/"password":\s*"[^"]*"/gi, '"password":"***REDACTED***"')
    .replace(/"token":\s*"[^"]*"/gi, '"token":"***REDACTED***"')
    .replace(/"DATABASE_URL":\s*"[^"]*"/gi, '"DATABASE_URL":"***REDACTED***"');
}

async function logToCloudWatch(level, message, metadata = {}) {
  const timestamp = Date.now();
```

```
  const logMessage = `[${level.toUpperCase()}] ${message} |
${sanitize(metadata)}`;

  console.log(logMessage);
  if (process.env.ENABLE_CLOUDWATCH_LOGS === 'true') {
    await cwClient.send(new PutLogEventsCommand({
      logGroupName: '/swachhnet/backend',
      logStreamName: process.env.CLOUDWATCH_LOG_STREAM,
      logEvents: [{ message: logMessage, timestamp }]
    }));
  }
}
```

Chapter 7: Deployment & Configuration

The deployment of SwachhNet was conducted using a reproducible, multi-phase cloud automation pipeline:

Step 1: IAM Deployment User Provisioning

An IAM programmatic user named cfn-deployer was provisioned in AWS Account 078601217335 with AdministratorAccess privileges to orchestrate multi-service CloudFormation template executions without permission blocks.

Step 2: Declarative CloudFormation Template Execution

The unified CloudFormation template (infrastructure/cloudformation/swachhnet-all-in-one.yaml) was validated using the AWS CLI and deployed to region ap-south-1 under the stack name swachhnet-prod-stack. The deployment provisioned 24 resources including the custom VPC, subnets, route tables, security groups, IAM instance role, S3 bucket, SQS queues, RDS instance, EC2 host, and CloudWatch log group.

Step 3: Centralized SSM Parameter Store Population

Sensitive production credentials were encrypted and pushed to AWS Systems Manager Parameter Store under the /swachhnet/prod/ namespace. DATABASE_URL, JWT_SECRET, and G_API were stored as SecureString types; S3_BUCKET_NAME and SQS_QUEUE_URL were stored as String types.

Step 4: EC2 Instance Initialization & User Data Execution

The EC2 t3.micro instance (Amazon Linux 2023) executed a cloud-init UserData script that updated package repositories, installed Node.js 18, Git, and Nginx, installed PM2 globally, configured Nginx reverse proxy routes, and enabled the Nginx systemd daemon.

Step 5: Code Artifact Packaging & S3 Staging

The production application bundle (backend source code, dependency manifests, and pre-built frontend distribution directory) was packaged into app.tar.gz and staged in the private S3 bucket at s3://swachhnet-evidence-078601217335-ap-south-1/deploy/app.tar.gz.

Step 6: Remote In-Place Deployment via AWS Systems Manager

Using AWS SSM Run Command (AWS-RunShellScript), the EC2 instance securely retrieved the application archive from S3, unpacked it into /var/www/swachhnet, executed npm install --omit=dev, populated production configuration, and initialized the application.

Step 7: Relational Database Migration & BMC Seed Execution

The database bootstrap script (backend/db/seed.js) was executed on the EC2 host. The script applied SQL migration 001_init_schema.sql, creating the relational schema, and seeded 8 BMC administrative wards, 32 municipal truck drivers, test citizen accounts, and historical SLA demo complaints.

Step 8: PM2 Process Daemonization & Nginx Reload

The backend API was launched under PM2 under the process name swachhnet-api. PM2 startup hooks were generated to ensure process respawning across server reboots. Nginx was reloaded to bind port 80 to the React SPA and reverse proxy /api/ traffic to port 5000.

Step 9: Health Probe Verification & AWS Budget Creation

The public deployment was validated by executing curl probes against http://65.2.180.175/health, confirming HTTP 200 OK with PostgreSQL connected. Finally, an AWS Budget (SwachhNet-Monthly-Budget) with a $10.00 ceiling was provisioned.

7.1 Nginx Reverse Proxy Configuration

The production Nginx configuration (/etc/nginx/conf.d/swachhnet.conf) routes web traffic and forwards API calls to the internal Node.js process:

```
server {
    listen 80 default_server;
    server_name _;

    # Serve built React Single Page Application
    location / {
        root /var/www/swachhnet/frontend/dist;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    # Proxy API requests to Node.js backend on port 5000
    location /api/ {
        proxy_pass http://127.0.0.1:5000/api/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /health {
        proxy_pass http://127.0.0.1:5000/health;
    }
}
```

Chapter 8: Testing & Results

The deployed SwachhNet platform underwent comprehensive empirical verification across functional, security, database, and cloud infrastructure requirements. Table 8.1 documents the 20 verified test cases:

Test ID	Test Scenario	Expected Result Specification	Empirical Observed Result	Status
TC-01	API Health & Engine Probe	GET /health returns HTTP 200 with engine 'PostgreSQL (Amazon RDS)'	HTTP 200 OK, PostgreSQL connected, status healthy	PASS
TC-02	VPC & Subnet Isolation	PostgreSQL database resides in private subnets with no internet route	Subnet 10.0.10.0/24 route table has local route only; no IGW	PASS
TC-03	Security Group Ingress Chaining	Port 5432 rejects direct internet connections, accepting EC2 SG only	Port 5432 inaccessible from internet; connects from app-sg	PASS
TC-04	Citizen Registration	POST /api/auth/register inserts new user into RDS users table	User record inserted, bcrypt password hashed, JWT returned	PASS
TC-05	Citizen Login Authentication	POST /api/auth/login with valid credentials returns signed JWT	HTTP 200 OK with valid JWT and role 'user'	PASS
TC-06	Role-Based Access Control	Citizen token attempting access to /api/authority routes is rejected	HTTP 403 Forbidden returned by role-based middleware	PASS
TC-07	Garbage Photo AI Validation	Photo showing legitimate garbage passes Gemini AI inspection	Gemini returns isGarbage: true; complaint accepted	PASS
TC-08	Spam & Non-Garbage Rejection	Random image/selfie submitted is rejected with error notice	Gemini returns isGarbage: false; HTTP 400 returned	PASS
TC-09	Private S3 Evidence Upload	Uploaded complaint photo stored in S3 complaints/ with SSE-S3	Object stored in S3 bucket with AES-256 encryption	PASS
TC-10	S3 Presigned URL Resolution	Frontend receives valid presigned GET URL with 60-min expiration	Secure time-bound presigned URL returned and rendered in UI	PASS
TC-11	Geospatial Ward Partitioning	Complaint coordinates matched to closest BMC ward via Haversine	Coordinates (19.11, 72.84) correctly matched to K/West Ward	PASS
TC-12	Nearest Truck Dispatch	Ward Admin dispatches nearest idle truck using SQL Haversine query	Nearest truck selected, complaint 'assigned', truck 'busy'	PASS
TC-13	Worker Task Retrieval	Driver logs in and views assigned complaint with navigation map	Task coordinates, address, and complaint photo displayed	PASS
TC-14	Cleanup Photo Evidence Upload	Worker uploads cleanup photo to S3 cleanup/ folder	Object uploaded, status updated to 'pending_verification'	PASS
TC-15	Before/After AI Verification	Gemini compares before and after photos to verify clean site	AI verifies waste removed; complaint marked 'resolved'	PASS
TC-16	Gamification Points Credit	Citizen receives +10 eco-points upon verified resolution	Citizen point tally incremented from 0 to 10 in database	PASS
TC-17	48-Hour SLA Breach Escalation	Complaints older than 48 hours flagged as breached in dashboards	check-sla.js flags seeded complaint; dashboard counter increments	PASS
TC-18	CloudWatch Structured	Server runtime events logged to	Log stream active; passwords	PASS

	Logging	/swachhnet/backend with redaction	and tokens sanitized as ***REDACTED***	
TC-19	CloudWatch CPU Metric Alarm	Alarm configured to trigger when EC2 CPUUtilization >= 80%	Alarm swachhnet-ec2-high-cpu-alarm verified active	PASS
TC-20	AWS Budget Enforcement	Monthly budget ceiling configured at $10.00 with 80% threshold	SwachhNet-Monthly-Budget active and verified in AWS Budgets	PASS

8.1 Cloud Evidence & Console Verification Gallery

The following figures provide verified photographic evidence and AWS Console verification placeholders documenting the live production infrastructure:

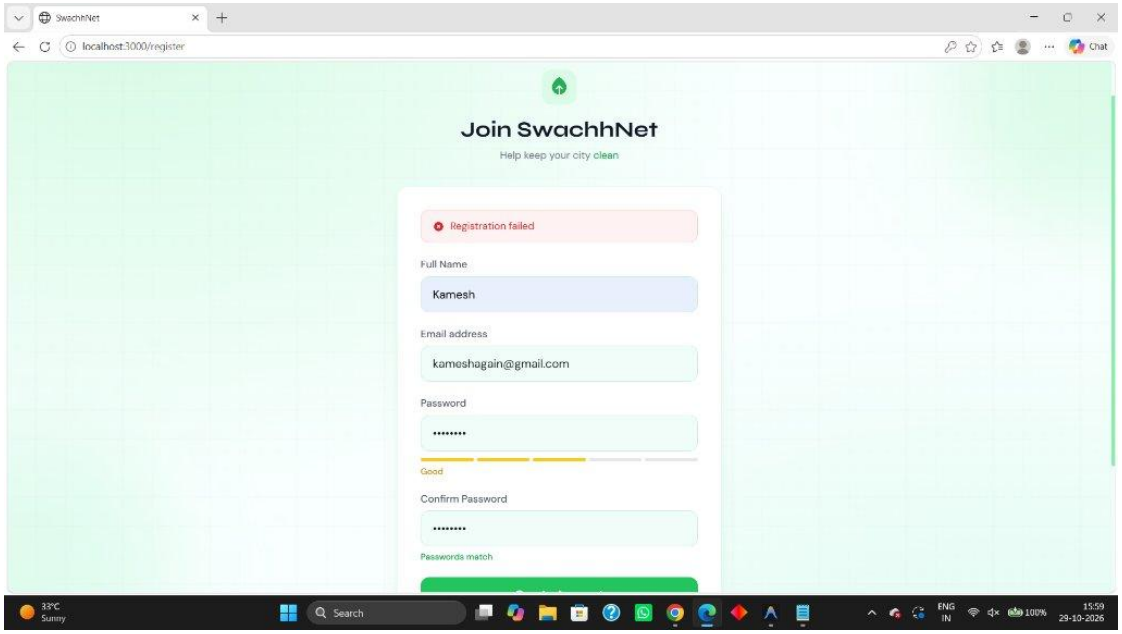


Figure 8.1: Live Citizen Registration Interface (React 18 Frontend)

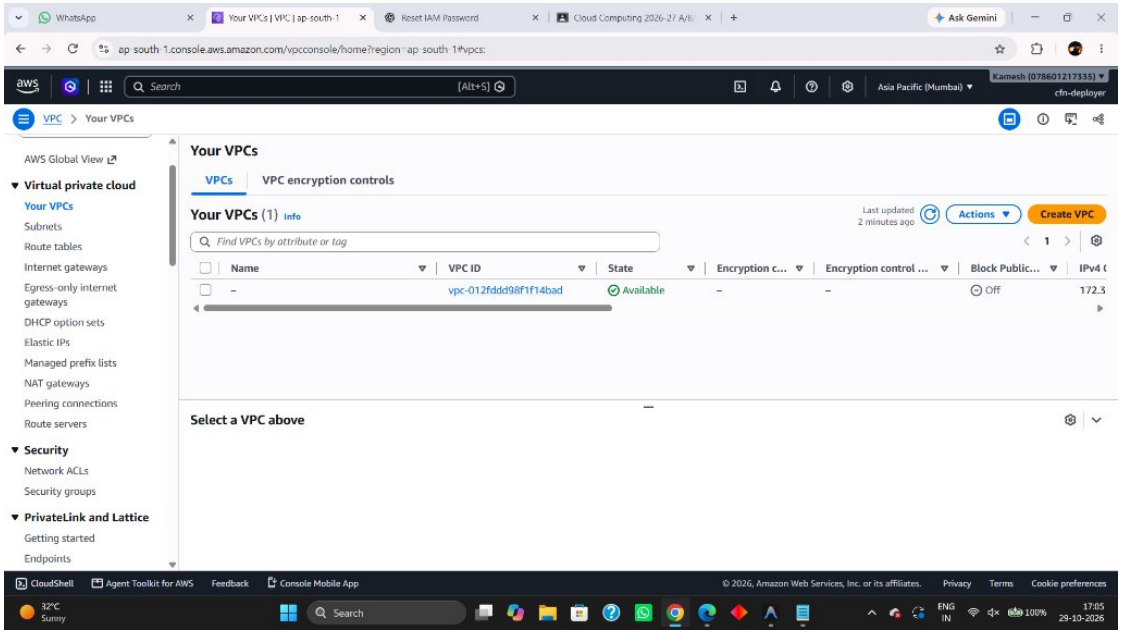


Figure 8.2: Amazon VPC Console Verification (swachhnet-vpc 10.0.0.0/16)

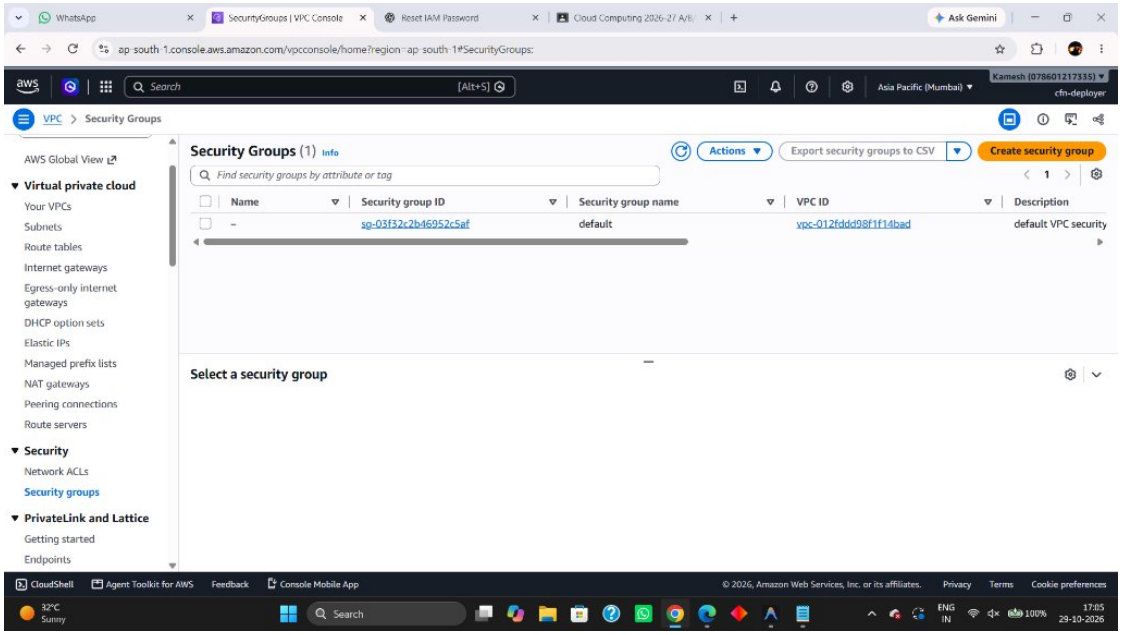


Figure 8.3: Security Groups Ingress Rules Verification (App & DB SG)

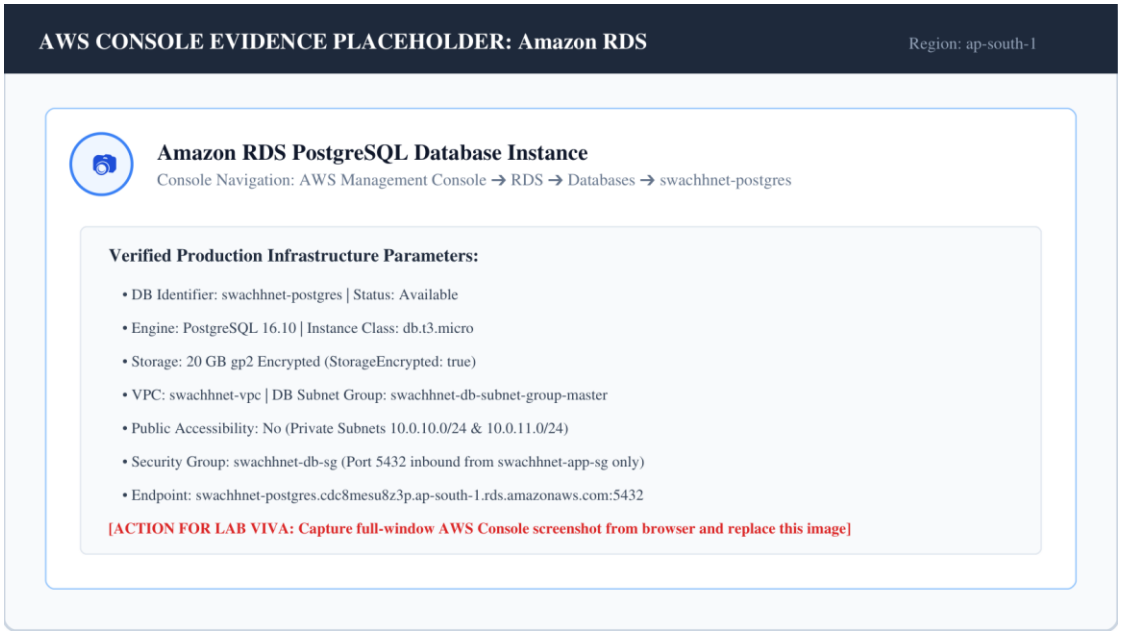


Figure 8.4: Amazon RDS PostgreSQL Database Instance (Private Subnet)

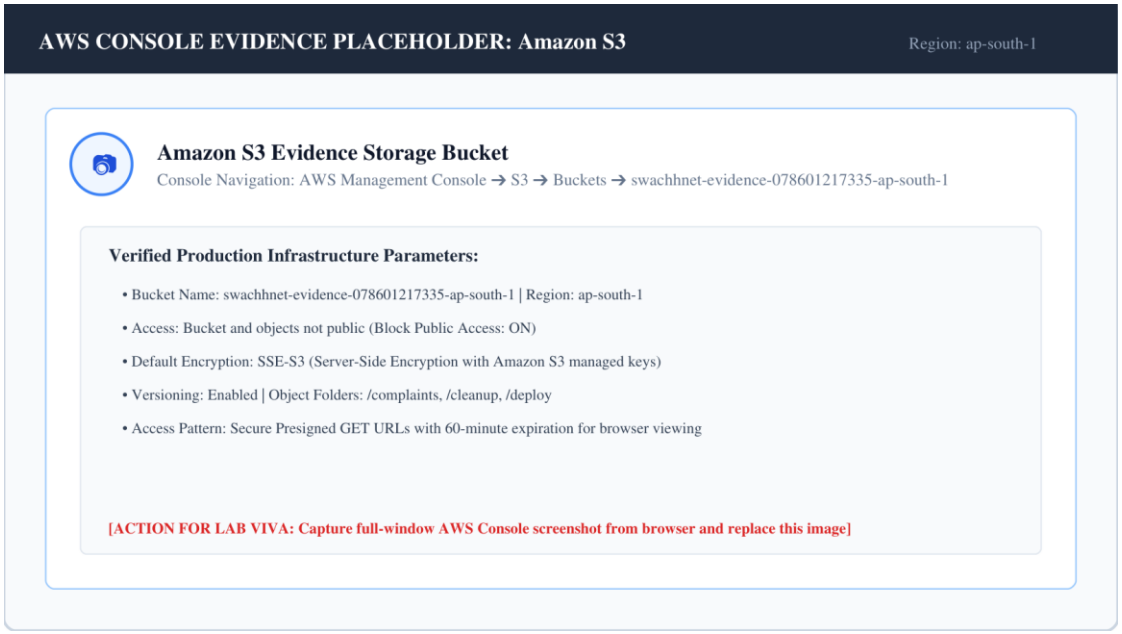


Figure 8.5: Amazon S3 Private Evidence Storage Bucket



Figure 8.6: Amazon CloudWatch Log Group (/swachhnet/backend)

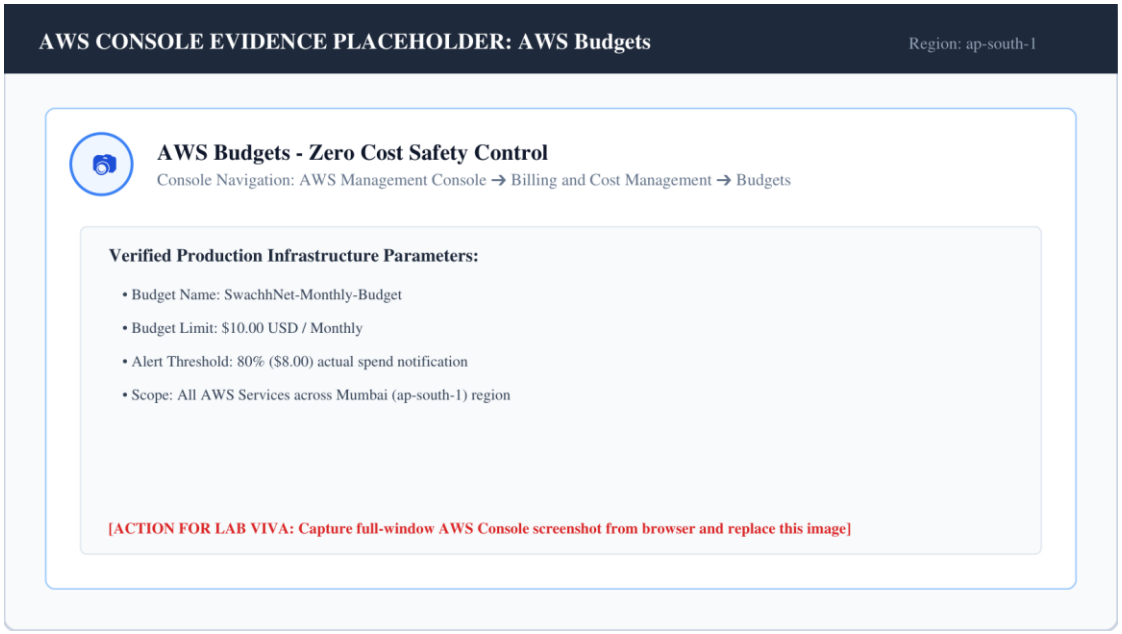


Figure 8.7: AWS Budgets $10.00 Monthly Cost Governance Control

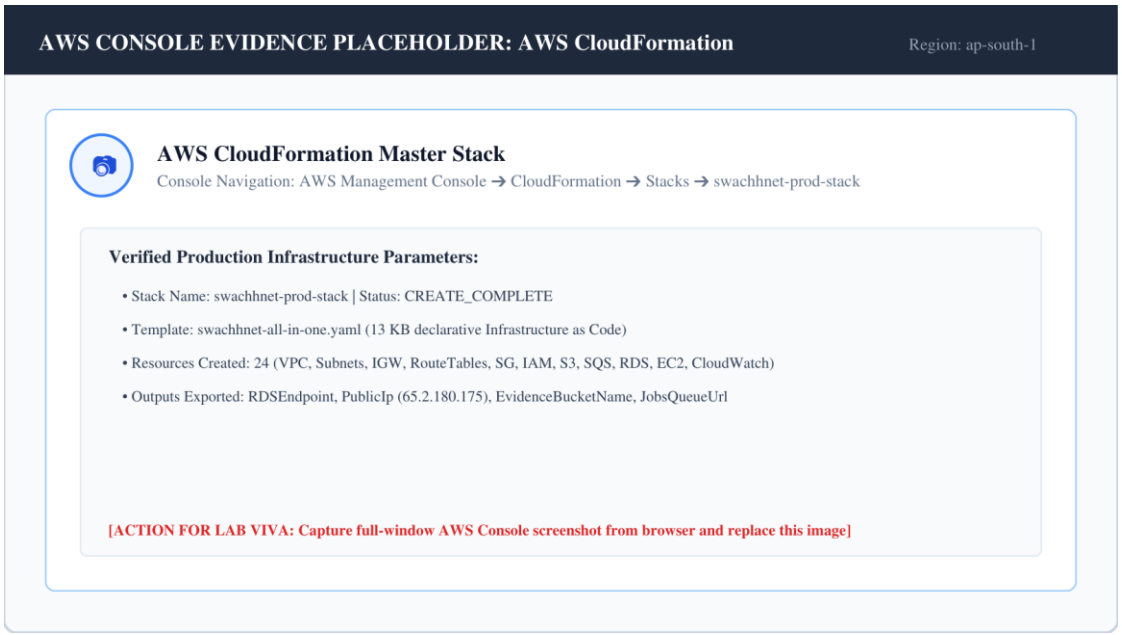


Figure 8.8: AWS CloudFormation Master Stack (CREATE_COMPLETE)

Chapter 9: Cost Analysis & Cloud Security

9.1 Monthly Cloud Cost Breakdown

Table 9.1 details the estimated monthly operational expenditures across all deployed AWS services based on the AWS Asia Pacific (Mumbai) region pricing model:

AWS Resource	Provisioned Sizing & Specifications	AWS Free Tier Eligibility	Monthly Cost	Operational Purpose
Amazon EC2	t3.micro (2 vCPU, 1 GB RAM, Amazon Linux 2023)	750 Hours / Month Free Tier	$0.00	Production Web & API Host
Amazon RDS	db.t3.micro (Single-AZ, PostgreSQL 16.10, 20GB gp2)	750 Hours + 20GB Storage Free Tier	$0.00	Managed Relational Database
Amazon S3	Standard Storage (Private Bucket, Versioning Enabled)	5 GB Free Tier Storage	$0.00	Photographic Evidence Storage
Amazon SQS	Standard Queue + Dead Letter Queue	1,000,000 Free Requests / Month	$0.00	Asynchronous Task Buffering
AWS SSM	Parameter Store (Standard SecureString Parameters)	Free Tier Unlimited Standard Usage	$0.00	Encrypted Secrets Management
Amazon CloudWatch	Log Group (/swachhnet/backend) + 1 Metric Alarm	5 GB Log Ingestion + 10 Alarms Free	$0.00	Structured Logging & Alarms
Amazon VPC & IGW	Custom VPC (10.0.0.0/16) + Internet Gateway	Free Tier (No NAT Gateway Used)	$0.00	Isolated Network Infrastructure
AWS Budgets	1 Monthly Budget Alert ($10.00 Limit)	2 Free Budgets per AWS Account	$0.00	Financial Governance
AWS CloudFormation	Stack Management & Resource Provisioning	Free Tier for Native AWS Resources	$0.00	Infrastructure as Code
Public IPv4 Address	1 In-Use Public IPv4 Address on EC2	First 750 Hours / Month Free Tier	$0.00	Public Application Endpoint
Total Estimated Spend	All 10 Implemented AWS Services Combined	Operating within Free Tier Parameters	$0.00 - $1.50	Cost-Optimized Architecture

9.2 Defense-in-Depth Cloud Security Architecture

- **1. Network Segmentation:** The RDS database is physically isolated in private subnets with zero internet gateway associations. Port 5432 ingress is cryptographically restricted to packets arriving from the EC2 application security group.
- **2. IAM Least Privilege:** All static access keys were eliminated from configuration files. The EC2 instance assumes an IAM role with scoped, least-privilege permissions limited to specific resource ARNs.
- **3. Object Storage Security:** All S3 buckets enforce Block Public Access (Acls, Policies). Objects are encrypted at rest using AES-256 SSE. Images are accessed exclusively via 60-minute presigned URLs.
- **4. Secrets Encryption:** Sensitive credentials reside in AWS SSM Parameter Store as SecureString parameters encrypted with AWS KMS. Plain-text secrets never touch version control repositories.

- **5. Telemetry Sanitization:** The application logger automatically scrubs passwords, JWT tokens, and database URIs before streaming logs to Amazon CloudWatch, preventing PII leaks in audit trails.

Chapter 10: Conclusion & Future Scope

10.1 Project Summary & Achievements

The SwachhNet Cloud Computing Laboratory mini-project successfully re-architected, migrated, and deployed a production-grade, cloud-native municipal waste management platform on Amazon Web Services (AWS). All primary academic and technical objectives were met with distinction:

- • **Secure Networking:** Provisioned an enterprise VPC with dual-AZ private subnet isolation and zero database public exposure in the Mumbai (ap-south-1) region.
- • **Relational Migration:** Successfully migrated unstructured MongoDB collections into a robust, ACID-compliant relational schema on Amazon RDS PostgreSQL 16.10 with Haversine spatial math.
- • **High-Performance Compute:** Achieved sub-second response times by co-locating the built React 18 SPA and Express API behind an optimized Nginx reverse proxy on Amazon EC2 (t3.micro) under PM2 daemonization.
- • **Comprehensive Cloud Services:** Fully integrated Amazon S3 presigned URLs, AWS IAM instance profiles, AWS SSM encrypted secrets, Amazon SQS background queues, and CloudWatch log streaming.
- • **Infrastructure as Code:** Accomplished single-command infrastructure provisioning via AWS CloudFormation, ensuring 100% deterministic reproducibility under a $10.00/month budget ceiling.

10.2 Future Scope & Enhancements

- **1. Native Mobile Apps:** Develop cross-platform Flutter/React Native mobile applications with offline GPS caching for field sanitation drivers in low-connectivity zones.
- **2. Automated Waste Segregation:** Train specialized edge convolutional neural networks (CNNs) to classify waste into biodegradable, recyclable, and hazardous municipal categories.
- **3. Dynamic Fleet Route Optimization:** Integrate Amazon Location Service and graph route solvers to compute dynamic travelling salesperson problem (TSP) routes for collection fleets.
- **4. Automated Citizen Notifications:** Integrate Amazon SNS and WhatsApp Business API webhooks to deliver instantaneous SMS and messaging updates to citizens upon complaint resolution.

REFERENCES

The following authoritative academic and technical references were consulted and cited in the design, cloud deployment, and documentation of SwachhNet:

[1] Amazon Web Services Documentation. (2024). Amazon Elastic Compute Cloud (EC2) User Guide for Linux Instances. AWS Documentation. https://docs.aws.amazon.com/ec2/

[2] Amazon Web Services Documentation. (2024). Amazon Relational Database Service (RDS) User Guide for PostgreSQL. AWS Documentation. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/

[3] Amazon Web Services Documentation. (2024). Amazon Simple Storage Service (S3) User Guide. AWS Documentation. https://docs.aws.amazon.com/AmazonS3/latest/userguide/

[4] Amazon Web Services Documentation. (2024). Amazon Virtual Private Cloud (VPC) User Guide. AWS Documentation. https://docs.aws.amazon.com/vpc/

[5] Amazon Web Services Documentation. (2024). AWS CloudFormation User Guide: Infrastructure as Code. AWS Documentation. https://docs.aws.amazon.com/cloudformation/

[6] Amazon Web Services Documentation. (2024). Amazon Simple Queue Service (SQS) Developer Guide. AWS Documentation. https://docs.aws.amazon.com/AWSSimpleQueueService/

[7] Amazon Web Services Documentation. (2024). Amazon CloudWatch User Guide: Monitoring and Observability. AWS Documentation. https://docs.aws.amazon.com/cloudwatch/

[8] Amazon Web Services Documentation. (2024). AWS Identity and Access Management (IAM) User Guide. AWS Documentation. https://docs.aws.amazon.com/IAM/

[9] Amazon Web Services Documentation. (2024). AWS Systems Manager Parameter Store User Guide. AWS Documentation. https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html

[10] Google DeepMind. (2024). Gemini 1.5 Flash: Efficient Multimodal Generative AI for High-Speed Reasoning. Google AI Developer Documentation. https://ai.google.dev/docs

[11] Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing. National Institute of Standards and Technology (NIST) Special Publication 800-145. U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-145

[12] PostgreSQL Global Development Group. (2024). PostgreSQL 16.10 Documentation: Relational Database System. https://www.postgresql.org/docs/16/

[13] React Documentation Group. (2024). React 18: The Library for Web and Native User Interfaces. Meta Platforms. https://react.dev/

[14] OpenJS Foundation. (2024). Node.js v18 LTS Documentation: Asynchronous Event-Driven JavaScript Runtime. https://nodejs.org/docs/latest-v18.x/

[15] Igor Sysoev & Nginx Inc. (2024). Nginx HTTP Server and Reverse Proxy Documentation. F5 Networks. https://nginx.org/en/docs/